

Changelog

All notable changes to **Rev80** are documented here. Sections dated before the 2026-08 rebrand describe the project under its former name, vibechecker, and retain it as an accurate record. Format follows [Keep a Changelog](#).

[Unreleased]

[0.1.2] - 2026-09-17

fix/stability-cluster (2026-08-30, merged 2026-09-11)

The four audit findings about the app *staying up* rather than measuring correctly: two criticals (S-01, S-02) plus the two that decide whether a crash leaves anything to read (H-07, H-03). Written against develop at 2026-08-30 and merged unchanged after the tachometer and raw-rate work landed; every defect below was still live at merge, and the merge needed no conflict resolution. **927 unit tests and all 23 hardware tests pass on the merged result** (4424A, AWG loopback).

Ordered deliberately: evidence first, because fixing S-01 and S-02 blind would mean that if the app still died there would still be nothing to read.

Added

- `logger.install_excepthooks()` (idempotent, replaces the bare `sys.excepthook = ...` line in both front ends) — three crash routes, each previously leaving a different amount of nothing.
 - `threading.excepthook` . `sys.excepthook` covers only the main thread, and everything interesting runs off it: the PicoScope poll thread, the simulation generator, the monitor writer, the autoconnect and reprocess workers. Those deaths went to stderr — nowhere, when launched from a desktop entry. Now routed to the rotating log, naming the thread and noting that anything waiting on it will hang rather than fail. Verified end to end: a raising worker writes a full traceback to `error.log`.
 - `faulthandler` → `log/faulthandler.log`, the only evidence that survives a SIGSEGV and what distinguishes a driver-level crash (X-05) from an OOM kill, which leaves nothing at all. Verified by deliberately dereferencing NULL: the file names the exact frame and lists the loaded extension modules.
 - **A periodic resource line** (`MonitorController`, every 5 minutes during a session): RSS, thread count, burst frame retention, writer queue depth. SIGKILL cannot be trapped, so the only possible evidence predates it — this turns an unexplained disappearance into a readable ramp, and it is exactly the ramp S-02 produces. It never raises: a diagnostic that takes down the session it is diagnosing is worse than no diagnostic.
- `tests/test_app_lifecycle.py` (177), `tests/test_bounded_resources.py` (233), `tests/test_crash_evidence.py` (150).

Fixed

- **S-01 (critical) — shutdown was skippable, and skipping it stranded the device.** Two defects that compound, and together explain both halves of the reported symptom: *“I came back and the session was truncated, and then the scope wouldn’t connect until I replugged it.”*
 - `GUI.cleanup()` never called `self._monitor.stop()`, and the writer is a **daemon** thread, so the interpreter killed it without unwinding — possibly mid- `h5py.File(..., 'a')` — with captures still queued. `MonitorController.stop()` already flushed the partial burst and drained the writer correctly; it was simply never reached on app close. `cleanup()` now stops the monitor **first**, then closes the device, then destroys the DPG context, each step individually guarded — a wedged writer must not prevent `ps4000a.closeUnit`, because a device left open is what makes the next launch fail with `PICO_NOT_FOUND`.
 - `GUI.run()`’s loop body had no `try/except` and `__main__.main()` had no `try/finally`, so any exception in the render path skipped `cleanup()` entirely. `main()` now wraps `run()` in `try/finally` and `cleanup()` is idempotent, so the loop’s own guarded exit is harmless. This also downgrades **X-02** — a shared `.h5` with `binsize=0` raising `ZeroDivisionError` in the render path — from a process-ending crash that strands the device to a logged error.
 - Render-loop policy, as chosen: log once per exception **type**, keep rendering, and give up after `MAX_CONSECUTIVE_RENDER_ERRORS` (30) consecutive failures by breaking the loop so shutdown still runs *through* `cleanup()`. Deduplication matters because a persistent fault would otherwise write a traceback at frame rate and roll every other diagnostic out of the rotating log — exactly the **S-09** failure mode. A successful frame clears the streak, so occasional bad frames over a long run cannot accumulate into a shutdown; per-type totals are logged once on exit.
 - `cleanup()` reads `_render_errors` through `getattr`: it runs from `main()`’s `finally` and must survive a GUI that failed partway through construction. It is the one method that cannot be allowed to raise, because it is what closes the device.
- **S-02 (critical) — every resource an unattended run can grow is now bounded.** Three defects compounding into the most likely way an overnight session dies, and the one leaving the least evidence: an OOM kill is `SIGKILL` — no traceback, no `atexit`, no log line. The app simply vanishes.
 - **(a) Burst retention had no cap at all**, holding every frame *and* every `ChannelResult` until the single flush. Now capped by `burst_frame_cap()`, derived from `max_burst_s` and the acquisition period rather than a magic number, with a floor so a pathological period cannot produce a zero-length burst. `_burst_frames` and `_burst_all_results` are trimmed **together**, because `_flush_burst` indexes them in parallel — trimming one alone would put every overall against the wrong waveform. Hitting the cap warns once.
 - **(b) `max_burst_s` was inert on the path that fires unattended.** It was enforced only inside `IntervalGate.enter_burst()`, which only the *manual* path calls; the anomaly path set `_burst_end_mono` directly. Both now go through a shared `capped_burst_end()`; `0/None` still means unset rather than zero-length. `IntervalGate._burst_start` is also initialised in `__init__` — the retrigger branch reads it, and it was one refactor from an `AttributeError` in the monitor’s hot path.
 - **(c) The writer queue was `queue.Queue()` with no `maxsize`**, so its `except queue.Full` branch was unreachable dead code and `enqueue()` always returned `True`. That return gates the capture counter, so **the UI reported successes that were never written to disk**. The queue is now bounded; on full it drops rather than blocks — back-pressure there would reach the render loop and stall acquisition behind the disk — and the drops are counted, logged with the queue depth, and surfaced in `status_snapshot()` as `dropped_captures`. A loss nobody can see is the same defect in a new place.

- **H-03 — a field log could not be tied to a build.** `__version__` now resolves from `git describe` in a source checkout, falling back to the stamped `_version.py` in an installed or frozen build. The stamp comes from a pre-commit hook that is **not** installed automatically, and was observed 100 commits stale.

Measured

- **Burst retention costs 2.76x the raw block**, measured on 4 channels through the real pipeline at `RAW_SAMPLERATE_HZ = 25600` (deep ndarray bytes reachable from one retained frame plus its `ChannelResults`):

binsize	acq. period	raw block	retained	multiple
0.5 Hz	2.000 s	1.638 MB	4.517 MB	2.76x
1.0 Hz	1.000 s	0.819 MB	2.259 MB	2.76x
2.0 Hz	0.500 s	0.410 MB	1.130 MB	2.76x

Uncapped growth is therefore **2.26 MB/s on 4 channels, ~8.1 GB/h, independent of both `F_max` and `binsize`** — since the raw/display split, stored frames are the fixed-rate capture, so a low `F_max` no longer buys headroom here the way it did when this defect was first written up (the original note cited ~2 GB at an `F_max` 50 kHz preset that no longer exists). At the shipped `max_burst_s` default of 600 s the cap holds one burst to ~1.36 GB. The docstring carries this table.

Notes

- All three fixes revert-checked — removing the monitor stop, the render-error deduplication, or `cleanup()`'s idempotence each makes tests fail.
- **The first revert-check pass found a hole worth recording:** reverting the writer queue to unbounded still passed all 15 tests, because the helper built its own queue with an explicit `maxsize` and nothing exercised `MonitorWriterThread`'s real `__init__`. Same shape as the power-vs-amplitude gap in feature/spectral-averaging. Tests that drive the real constructor were added; the revert now fails as it should.
- **H-01 is not addressed** — `_build_anomaly_hook` is still copy-pasted between `gui.py` and `headless.py`.

feature/tachometer (R43) (2026-09-01)

Added

- **rev80. tach** — tachometer edge detection and shaft-speed estimation. Pure functions plus two frozen dataclasses (`TachSettings`, `TachResult`), free of `dearpygui` / `h5py` / `DataCollector` imports. Nothing wires it in yet; this is step 1 of the tachometer feature.
 - `detect_edges()` — vectorised Schmitt trigger with sub-sample interpolation of the crossing instant. The vectorised form is not just 68x faster than the obvious loop (0.090 ms vs 6.1 ms on a 1.0 s block) but more correct: it requires a real crossing, where the loop reports a phantom edge at sample 1 whenever a block opens part-way through a pulse.
 - `estimate_rpm()` — **median of intervals**, not first-to-last. Measured at 1800 RPM over 200 reps, one dropped edge costs first-to-last 62.09 RPM and the median 0.15 RPM. Robustness to a miscount is worth more than tightness under jitter, because jitter shows up in `interval_spread` and a miscount does not.
 - Quality is classified, not collapsed: `ok` / `no_signal` / `too_few_edges` / `inconsistent` / `unsteady`. **rpm is None when there is no usable reading and never 0.0** — “I cannot see a tach signal” and “the shaft is stopped” send an analyst to different places.

- **Channel roles and the speed gate** (`AcquisitionSettings`) — step 2, still unconsumed.
`channel_roles` ({ch: 'vibration'|'tachometer'}) with `role_for()` , and the derived `tach_channels` / `vibration_channels` partitions that `process_samples` will iterate. An unrecognised role string falls back to `vibration` rather than propagating, so a hand-edited YAML cannot invent a third channel kind that every downstream branch then fails to handle; and the partitions filter by *enabled* channels, so a tach role left on a switched-off input does not have the collector hunting for a pulse train nobody is sampling.
- **speed_gate_enabled** / **speed_gate_rpm** / **speed_gate_tolerance_pct** — off by default, since with no tachometer fitted there is no reference to gate against. `speed_gate_rpm = None` means “latch from the first valid frame” and survives the config round trip, for the same reason `band_fmin` / `band_fmax` do: writing a resolved value back would freeze one session's running speed into the config.
- **role** and a nested **tach** block in `_BUILTIN_CHANNEL_TEMPLATE` , and the three `speed_gate_*` keys in `_BUILTIN_ACQ` , so every device and acquisition YAML written before R43 upgrades silently through the existing merge.
- **Hardware close-out on the 4424A** — 14 new self-skipping tests in `tests/test_picoscope_hw.py` , all through the real acquisition path (`PicoScopeStream` → `antialias_decimate` → `receive_data` → `tach_result`) rather than handing synthetic arrays to the detector. **23 hardware tests pass.**
 - AWG sweep 300–10200 RPM, every point within the published $\pm 0.2\%$ of reading.
 - The reported rate is asserted to be 41666.5 Hz and *not* `RAW_SAMPLERATE_HZ` — the 4.166% trap that is exactly right in CI and wrong on hardware.
 - The AC-coupling failure reproduced electrically with a 70%-duty arbitrary waveform: a fixed threshold returns no reading while adaptive tracks the shaft.
 - Duty cycle measured against the generator's known 50%.
 - A tach channel confirmed to produce no `ChannelResult` on real hardware.
 - Four channels with a tach streaming 8 s with zero overflow and zero rate-degradation events.
 - One test was written wrong and corrected: a fixed threshold at **50% duty** is the one case where fixed and adaptive coincide, and whether a 1000 mV level lands inside the AC-coupled swing was observed both ways across runs. Pinning either outcome would have pinned a coin-flip, so the assertion now covers only the regime where the difference is real and repeatable.
- **Tachometer tab, and selectable rotation-rate units** — step 9.
 - A **Tachometer tab** in the config dialog, which *owns* the tach role: channel claim, polarity, threshold mode and level, minimum amplitude, reflector size, rate units, a live waveform with the threshold drawn and detected edges marked, and RPM / quality / duty / span readouts. Tach setup is a commissioning activity done once per installation, so the diagnostic view belongs where the settings are — adjust, watch the edges move, confirm the rate, all on one screen. Closing the dialog leaves only the derivatives, which solves “hide the waveform once it works” structurally rather than with a toggle the operator has to manage. (Verified first that a dearpygui modal does not block the render loop: 60/60 frames advanced with a modal shown.)
 - The waveform is **aligned so the first detected pulse sits at t = 0**. A free-running trace jitters by up to a whole period between frames; aligned, successive frames overlay and a threshold adjustment is legible. X limits span two periods either side.
 - The **Channels tab no longer has a role control**. A claimed channel is shown read-only in its summary line — its settings there are meaningless, since a pulse train has no sensor, engineering unit or amplitude mode, and two screens able to set the role could disagree.
 - **Rotation-rate units**: RPM, Hz, rad/s (ω), deg/s — `rad/s` is angular frequency, so it is one option labelled with both names, not two. The unit is rendered wherever a rate appears: 30 is a plausible RPM, a plausible Hz and a plausible rad/s, and they differ by factors of 60 and 6.28. It is a **display**

- preference only** — `TachResult` and every stored file stay in RPM, because a number whose meaning depends on a setting is the class of defect this codebase keeps finding.

 - The slowest measurable shaft for the current bin size is shown as **information, not validation**: an operator must be able to configure the tach against a machine that is not running, using their best guess.
- **1× shaft-rate marker and level** — the first consumer of shaft speed in the display, and deliberately the simplest: a vertical line on the spectrum at 1×, and the level beside it on each channel result card above the peaks table. No resampling, no interpolation, no second axis type.

 - A line's frequency is only diagnostic relative to 1×: unbalance sits on it, misalignment on 2×, and a bearing tone characteristically *between* orders.
 - `ChannelResult.one_x_hz / .one_x_amplitude` . 1× rarely lands on a bin centre, so the level is the **larger of the two bins straddling it** — which recovers most of what a strictly-nearest-bin reading loses to the offset, and keeps the rule consistent with how peaks are already reported.
 - Both are **hidden, not zeroed**, when there is no tachometer reading, and the amplitude is `None` when 1× falls outside the displayed band: `F_max` can sit below the shaft rate on a fast machine, and reporting the edge bin would be a wrong number rather than a missing one.
- **Duty cycle and pulse widths** on `TachResult`, persisted in v5. `detect_edges` found only the active edge, so pulse width — and therefore duty — was not captured at all. `detect_pulses()` now returns both edges from the same Schmitt state, so the two can never disagree about where the signal was high, and `pulse_widths()` pairs them.

 - A pulse straddling either block boundary is **dropped, not truncated**: its remainder is a function of where the block happened to start, so including it would bias duty by something that has nothing to do with the reflector.
 - Duty is measured against the *pulse* period rather than the shaft period, because a reflector subtends a fraction of the interval between pulses whenever there is more than one per turn.
 - Under `falling` polarity it measures the notch, which is what a keyphasor's key actually subtends.
 - This is the prerequisite for surface velocity (R46): the reflector subtends `duty` of a revolution, so circumference is `L/duty` and `v = f · L/duty` — the tape doubles as a shaft-diameter measurement. Rolled into v5 rather than a new file version, since the format has not shipped and no real-world file contains a `TachResult` yet.
- **RmsThresholdHook default 10% → 50%** (decision D-1). Not a tachometer change — a correction to a shipped default that the speed analysis exposed. For a rigid rotor below its first critical the 1× velocity goes as ω^3 :

speed deviation	1× velocity change
0.5%	+1.5%
1.0%	+3.0%
2.0%	+6.1%
3.2%	+10.0% ← the old default
5.0%	+15.8%
10.0%	+33.1%

A 3.2% speed change crossed the threshold on its own, and a typical induction motor's ~2% no-load-to-full-load slip swing shows up as a +6% rise on a machine whose condition has not changed. On any VFD or load-following machine the detector was measuring load. 50% is the level at which a broadband RMS rise means something without a speed reference. Where a tachometer is fitted the speed gate is the more certain discriminator and this can be tightened per installation; where one is not — common, and often

impractical to retrofit — detection has to come from envelope techniques or fixed thresholds instead. Changed in all five places that carried it (the hook, the config template, and both `_build_anomaly_hook` copies plus the headless summary), so `tests/test_anomaly_hook_build.py` still passes.

- **Monitor sessions record shaft speed, `session.h5` `_FILE_VERSION 5` → `6`** — step 8. Every interval capture and burst now carries `rpm` and `speed_ok`. Without it a monitor trend point cannot be compared with another taken at a different load — the same argument the declared band already makes for the overall, and the prerequisite for any later order analysis.
 - A **scalar**, not a `{ch: rpm}` map: this instrument supports one tachometer on one shaft (multi-shaft needs order ratios and a machine-train model, a different feature), so every result in a frame carries the same reading. NaN means no reading, never 0.0.
 - `MonitorController._compute_pretrigger_overalls()` now **skips tachometer channels**. It reads `overall_ampl_by_integration_order`, which is never populated on a channel `process_sample` never runs on — so a tach was contributing a literal `0.0`, and `load_monitor_session()` then rebuilt a trend line pinned at zero for it.
- **rev80-headless refuses tachometer-role channels** — step 7. Tachometry is out of scope there (tracked as R44), but out of scope has to mean “does not do it” rather than “does it wrong”. Headless reads the same `devices/*.yaml` the GUI writes, so a channel the operator configured as a tachometer would otherwise be enabled, high-passed, given an overall, trended and fed to the anomaly hooks as vibration. Measured on a 5% duty pulse train at 1800 RPM through the real `process_sample`: overall 1514.9 mV, crest 5.00, **kurtosis 15.94** and 63 spectral peaks — an analyst reviewing that unattended session concludes a bearing is failing badly. It also drifts on nothing: a tach LED ageing from 5.0 V to 4.5 V moves that channel's overall by exactly −10%, the shipped `RmsThresholdHook` threshold, on three consecutive frames. The refusal logs at INFO rather than passing silently. The per-channel config loading moved out of `run()` into `_apply_channel_config()` in the process.
- **Speed gating** — step 6. A frame captured outside a declared shaft-speed window is still measured, displayed and stored, but is excluded from trending, baseline adaptation and alarm evaluation, because its amplitude is *correct* and simply not comparable. For a rigid rotor below its first critical the $1\times$ velocity goes as ω^3 , so a **3.2% speed change alone moves the overall 10%** — the shipped `RmsThresholdHook` default. On any VFD or load-following machine the anomaly detector has been measuring load rather than condition.
 - `ChannelResult.rpm / .speed_ok`; `speed_ok` defaults `True` so every existing construction site, fixture and reconstructed result is untouched.
 - `DataCollector.speed_ok()` is the single place the gate is evaluated, and `monitor/anomaly.valid_results()` the single place it is applied — a seam that already existed to answer exactly this question and is already called by every hook and both baseline-adaptation paths.
 - **Fails closed** on a missing reading: if the tach dies mid-session (cable pulled, tape peeled, LED aged out), treating “no speed reading” as “speed is fine” would leave an unattended monitor alarming on load swings it can no longer see — the exact false-alarm mechanism the gate exists to remove.
 - `speed_gate_rpm = None` latches the reference from the first frame that actually has one; an unreadable frame cannot latch.
 - **`_build_anomaly_hook` is untouched in both copies** and `tests/test_anomaly_hook_build.py` passes unmodified. That is the check that the seam was chosen correctly: adding the gate as a hook parameter would have meant editing both copies, which is what audit H-01 exists to prevent.

• **Tachometer persistence, measurement file `_FILE_VERSION 4` → `5` — step 5.**

- A tach channel stores `edge_times`, not a `data waveform` (decision D-2): ~30 float64 per second against 41666, a factor of ~1400, which matters most on the long unattended sessions where a tach channel would otherwise dominate the file. What is traded away is re-thresholding after capture; what is kept is everything that makes RPM a *view* on stored data — `pulses_per_rev` is a post-hoc divisor on the intervals, and a shaft-angle vector, if ever wanted, is an interpolation of the same edge times. Readers must branch on the presence of `data` rather than assume it.
- `/metadata/channels/{ch}` gains `role` and the `tach_*` calibration that produced the stored reading; `/frames/{i}/{ch}` gains `rpm`, `quality`, `n_edges`, `interval_spread` and `speed_drift_pct` as a cross-check; `/tach_trend/{ch}` holds the RPM history.
- **A newer file version now warns instead of being misread.** An older build reading a v5 file takes its most permissive branch and restores a tach as an ordinary vibration channel, computing a bogus overall on a square wave and trending it. The guard is worth having independently of the tachometer.
- The frame cache stays homogeneous — every entry is still a `VibeSample`, carrying an empty waveform and a populated `TachResult`, because dozens of consumers index it.
- `reprocess_session_trend()` skips tach channels, which would otherwise get a fabricated amplitude trend pinned at zero (`overall_ampl_by_integration_order` is never populated on a channel `process_sample` never runs on). Deferred here from step 4 because it needs the role restored from file metadata to know what to skip.

• **Tachometer channels wired into `DataCollector` — step 4.**

- `receive_data()` branches on `role`. A tachometer channel **skips the high-pass entirely** and is edge-detected in its place. Measured: the filter's overshoot on each falling edge re-crosses the threshold, turning 31 edges into 108 at 15% duty — an 1800 RPM shaft reads 6270. Detection runs here for the same reason the filter does: this is the one place a block arrives exactly once and in stream order.
- `tach_settings` / `set_tach_settings()` / `tach_settings_for()`, keyed like `scope_sensors`; `tach_for()` recomputes when calibration changed after capture, so RPM stays a *view* on stored data rather than a value baked in at capture time.
- `current_rpm()` — the displayed frame's shaft speed, or `None`. Never `0.0`.
- `process_samples()` now iterates `config.vibration_channels`. A tachometer produces no `ChannelResult` at all, which is what stops kurtosis 15.94 appearing on a channel card for a square wave.
- `tach_trend` / `get_rpm_trend()`, kept separate from `trend` because shaft speed must never pass through `UNIT_TO_SI`, `amplitude_scale` or `integration_steps` — the same reason crest factor sits beside `orders` rather than as a column of it. `get_trend_for_display()` and `init_trend_channels()` are partitioned to match.
- `eu_scaled_raw()` **raises** on a tachometer channel. With no `ScopeSensor` it would otherwise divide by a sensitivity of 1.0 and hand back raw mV labelled as engineering units — the classic field error, and one that looks entirely reasonable on screen.
- `reset_channel_config()` prunes `channel_roles` and `tach_settings`, so moving from a 4-channel scope to a 2-channel one cannot leave channel 3's tach role attached to an index the new device uses for vibration.
- `VibeSample.tach` / `_tach_config_key`, cached like `psd_mv` and `filtered_mv`.

• **Simulated tachometer signals and per-channel simulation sources — step 3.**

`SimulatedSensor._sample()` previously tiled one generated signal across every enabled channel, which

made a simulated tachometer impossible: the tach input would carry the same accelerometer waveform as the vibration input, so tach had no offline CI at all.

- `SimulatedSensor.channel_sources` — `{ch: (fn, *args)}` per-channel overrides. Empty keeps the tiled behaviour **byte-identical**, so no existing test or caller moves.
- `GenerateTachPulse()` — a pulse train in mV with a **finite rise** (1.5 samples by default). An ideal rectangle is a degenerate stimulus: no sample lands in the detector's hysteresis band and sub-sample interpolation has nothing to interpolate, so generating one would have CI exercise a regime the instrument never sees. Real edges arrive with about one intermediate sample, measured on a 4424A.
- `GenerateMachineWithTach()` — a vibration channel and a tach channel from the *same* shaft, sharing `running_rate` and an explicit shaft phase so the tach edge marks the angular position where the load zone peaks. The coherence is the whole point: a tach not locked to the vibration's own shaft rate cannot validate anything, because a broken tachometer and a correct one both return a plausible number against an unrelated signal — the same argument this module already makes about pure cosines being unable to validate envelope analysis. Pinned by a test asserting the measured RPM matches the 1× peak found in the vibration channel's own spectrum.
- `machine_with_tach_sources()` — the streaming counterpart, so the two rates cannot be set independently and drift apart.
- `GenerateBearingVibration()` gains an optional `shaft_phase` ; when omitted it is drawn from the same rng as before, leaving the existing draw order and output unchanged.

Changed

- **Tachometer support is specified around 1 pulse/rev** (decision D-6). The UI will offer no pulses/rev control: one reflective tape or one keyway is not merely the common installation but the accurate one. At 1 ppr every interval is exactly one shaft revolution, so encoder division error and once-per-rev speed modulation cancel *inside each interval by construction*; above 1 ppr they cancel only after a whole revolution has been observed, and extra pulses buy nothing before that. Measured, a 60-line encoder with $\pm 0.05^\circ$ division error and 0.5% once-per-rev modulation over 40 random start phases: 0.580% error at 0.05 rev, 0.344% at 0.25 rev, **0.091% at 1.00 rev and flat thereafter out to 20 rev**. The same test at 1 ppr gives **0.0013%** from three edges — 70× better than 60 ppr reaches at any window length.
 - `pulses_per_rev` and the divide are retained so the capability can be restored, and a value other than 1 is **honoured with a warning, never silently clamped** — clamping would report an integer multiple of the true speed with nothing on screen to say so.
 - Consequently there is **no minimum-revolutions constant and no new user config**: `MIN_EDGES = 3` already is two whole revolutions at 1 ppr.
 - Rejected: inferring ppr from multi-modal pulse periods. Unequally spaced reflectors are already flagged `inconsistent` by `interval_spread` once the two gaps differ by more than $\sim 90^\circ$ of shaft rotation, and a near-evenly-spaced pair reads an exact integer multiple — the most obvious possible error to a technician who knows the machine.
- The published slowest-measurable-shaft figures are **3× higher than first stated**. Guaranteeing three rising edges regardless of start phase needs a block spanning three periods, so the floor is `180/T_block` RPM: 45 RPM at 0.25 Hz bins, 180 at 1 Hz, and **1800 at 10 Hz** — where nothing below 1800 RPM can be read at all. That is the one case where a legitimate setup returns no reading, and the GUI must say so.

Fixed

- `AcquisitionSettings.copy()` carries `channel_roles`. The per-channel dicts live in the `channels` config section, so unlike the scalars they are outside the `to_dict` / `from_dict` round trip and are enumerated by name in an explicit tuple. A sixth dict added without extending that tuple is **audit H-08**

again, and for roles the silent result is a copy in which every tachometer has reverted to vibration — the pipeline then high-passes a pulse train and reports kurtosis ~16 on it. Pinned by a revert-checked test.

Measured

Constants carry the table that justifies them, per house convention. Verified on a PicoScope 4424A (serial 12462/0067) with AWG loopback on channel A.

- **The reported sample rate is 41666.5 Hz, not `RAW_SAMPLERATE_HZ` (40000).** The driver rounds the sample interval to 12 us, so the true rate is 83333/2 Hz. Any RPM computed from the constant reads **4.166% high on hardware and is exactly right in CI** — the worst combination a defect can have. `tach` takes the rate from the sample; a test pins it.
- **`MIN_PULSE_AMPLITUDE_MV` = 1000.0** — measured front-end noise with the AWG idle: 0.37 mV RMS at +/-1 V rising to 5.09 mV RMS at +/-20 V, worst block span 42.5 mV. The gate clears that by 23.5x and still sits a factor of two below any real logic-level swing. Without it an adaptive threshold on pure noise returns ~9100 edges/block — 547752 RPM.
- **Adaptive thresholding is the default, and the reason is electrical.** AC coupling removes the mean, and on a pulse train the mean is the duty cycle. On the bench at 30 Hz, a fixed threshold placed at the correct DC midpoint detected nothing at all above ~55% duty (AC-coupled signal maximum falls to 980 mV at 70% duty, 750 mV at 85%), while adaptive returned 1801.7 RPM in all ten duty/coupling combinations. The fixed-threshold failure is silent and reads as a stopped machine.
- **RPM accuracy: +/-0.2% of reading**, 300 to 10200 RPM at 1 pulse/rev (worst case 0.164% at 300 RPM, 0.040% at 10200). The earlier simulated claim of a fixed +/-0.2 RPM does not survive contact with hardware — error scales with speed.
- **`SPEED_DRIFT_MAX_PCT` = 1.0** — within-block speed change above which a frame is flagged `unsteady`. Bearing analysis is performed at steady state, so a smeared spectrum is rejected rather than corrected; this is what replaces an order-resampling path. The only constant in the module still set from simulation rather than the bench.

Notes

- Sub-sample interpolation only works on a **band-limited** edge. On an ideal rectangle both straddling samples sit at the rails and the estimator degenerates to nearest-sample quantisation. Real edges are band-limited by the mandatory anti-alias filter upstream, which is why bench accuracy beats what a synthetic square wave achieves — and why accuracy tests must push their signals through `antialias_decimate`.
- All 44 tests in `tests/test_tach.py` were revert-checked against 11 invariants. The first pass found **5 of them pinned by nothing** — the min-span gate inside `detect_edges`, hysteresis, the Schmitt low-state requirement, sub-sample interpolation, and polarity inversion could each be deleted with the suite still green. All five were tests passing for the wrong reason, and the cause was one shared assumption: an *ideal rectangle* transitions in zero time, so no sample ever lands in the hysteresis band and interpolation has nothing to interpolate, while rate alone is polarity-invariant so comparing rates proves nothing about which end of the pulse is being timed. Fixed by adding a band-limited pulse generator (`make_ramped_pulses`) and asserting edge *instants* rather than only rates. All 11 invariants are now pinned.

hotfix/RAW_SAMPLERATE (2026-09-09)

Fixed

- **The Acquisition dialog advertised a sample rate the instrument never produced.**
`RAW_SAMPLERATE_HZ` was lowered to relieve GUI lag while streaming 4 channels, which exposed a latent

defect in the display-rate derivation. `samplerate` was `nextpow2(2.56 * maxfreq)` — rounding *up* to a power of two, so it overstated the rate by up to 2x. That was invisible only while `raw_samplerate` was large enough to absorb the overshoot. Measured, at the 10 kHz preset against 25.6 kHz of acquisition:

	dialog showed	pipeline delivered
sample rate	32.8 kS/s	25.6 kS/s
lines	10001	computed from a rate that did not exist

It failed silently because `decimate_to_rate` returns the block untouched when `target_rate >= raw_rate`, so the overstated rate produced no error — just a readout that disagreed with the data, and `n_fft_bins / binsize_actual` derived from the fictitious rate.

`samplerate` is now **exactly** `2.56 * maxfreq`. This cannot overshoot: `maxfreq`'s setter already clamps to `raw_samplerate/2/1.28`, which is the condition `2.56 * maxfreq <= raw_samplerate`. A power-of-two *rate* bought nothing — the FFT length is `blocksize`, not the rate. Every preset rate is now 5-smooth and divides `raw_samplerate` exactly, so raw → display decimation is an exact integer factor (50/20/10/5/2/1) at all six presets.

- **blocksize is now `ceil(samplerate / binsize)`** rather than `nextpow2(...)`. With a power-of-two rate that divided exactly and a frame really was `1/binsize` seconds; with an exact-2.56x rate `nextpow2` would have made frames up to 2x longer than the dialog claims. Measured worst-case frame-length overshoot across the 54 preset combinations: **56.2% → 17.2%**, with 24 combinations now exact. The delivered bin is still never coarser than the requested one. `blocksize` is no longer a power of two, which does not matter: it is a Welch segment length, `pocketfft` is efficient for any 5-smooth length, and every preset combination is one.
- **RAW_SAMPLERATE_HZ = 25600** (2.56 × the 10 kHz top preset), replacing a briefly-set `10_000` that dropped the 2.56 factor. At 10 kHz raw the `maxfreq` setter silently clamped to 3906 Hz — the 5 kHz and 10 kHz presets were unreachable — and envelope bandwidth was halved. 25600 Hz gives `osr=3` (76.8 kHz/channel at the ADC), which asks *less* of the ADC than the 40000 Hz configuration measured clean on a 4424A. **Validated on hardware 2026-09-09** (4424A s/n 12462/0067): `effective_osr=3`, actual raw ADC rate 76923 Hz/channel against 76800 requested (+0.16%, the driver's discrete timebase), **0 overflow and 0 rate-degradation transitions** over 45 s sustained at both 3 and 4 simultaneous channels; all 9 AWG-loopback hardware tests pass. The GUI-load problem that motivated lowering the rate is separate and still open.
- **tests/test_picoscope_hw.py asserted a sample rate the stream never requested.** `STREAM_SAMPLERATE = 50_000` predated the raw/display split: `PicoScopeStream` acquires at `raw_samplerate`, so 50 kHz was never asked of the hardware. The assertion allowed 40% deviation — wide enough to hide the rate being wrong by a factor of 1.56 — and passed only while `raw_samplerate` happened to be 40 kHz, 20% away. At 25600 Hz it failed at 48.7%. Separately, `test_stream_start_stop_cycle` computed its sleep as `STREAM_BLOCKSIZE / STREAM_SAMPLERATE = 1.0 s` against a real `acquisition_period` of 1.95 s, so no callback could arrive and it reported a streaming failure that was its own. Both now derive from the config (`raw_samplerate`, `acquisition_period`) and the rate tolerance is 5%, against a measured quantisation error of 0.16%.

Changed

- `tests/test_acquisition_settings.py`: `test_samplerate_is_power_of_two` and `test_blocksize_is_power_of_two` pinned the exact behaviour that was wrong. Replaced with the invariants that matter — `samplerate` is exactly 2.56x `maxfreq`; the display rate never exceeds the

acquisition rate; the decimation ratio is an exact integer; no preset is clamped; the delivered bin is never coarser than requested and a frame is never more than one sample longer than `1/binsize`.

- `tests/test_declared_band.py`: the out-of-band tone was 1500 Hz against `F_max = 1000`, which sat in the guard band only because `nextpow2` inflated `fs/2` to 2048. At the correct `fs/2 = 1280` it is above Nyquist, where the decimation filter removes it outright (−240 dB measured) — the test would have passed without the band mask doing anything. Moved to 1100 Hz, measured to survive decimation at −0.7 dB, so the band mask is the only thing that can exclude it. Revert-checked: patching out the `band_fmax_resolved` clamp makes it fail by +123.6%.
- **Repo-relocation breakage (tooling only, no measurement impact)**. The checkout has moved three times (`~/CODE/reveng/vibegui` → `~/Documents/reveng/code/vibegui` → `~/Documents/reveng/vibration/rev80`) and each move stranded absolute-path state that fails *silently*:
 - `core.hooksPath` still pointed at the previous clone's `.git/hooks`, a directory that no longer exists. Git runs no hooks at all in that state, so the blocking `ruff check src/ tests/` gate and the `doc/*.pdf` re-render had not run since the move. Now set to the relative `.githooks`, which survives any future relocation. The stale `gitflow.path.hooks` (pointing two moves back, at `~/PurpleDocs/...`) was unset.
 - The editable install's `.pth` pointed at the dead `.../code/vibegui/src`, so `import rev80` raised `ModuleNotFoundError` and the `rev80 / rev80-headless` console scripts and the desktop launcher were all dead. Reinstalled editable. A stale pre-rename `vibechecker` distribution — a separate dist that `pip install -e .` does not touch — was uninstalled alongside it.

This went unnoticed because **pytest is immune to it**: `pyproject.toml` sets `pythonpath = ["src"]`, resolved from `rootdir`, so all 737 tests collected and passed against the source tree while every installed entry point was broken. Green CI does not prove the app launches.

- `.python-version`, `.vscode/` and `.ruff_cache/` are now gitignored (and `.python-version` untracked) so each checkout owns its own dev environment. Consequence: a relocated checkout no longer auto-selects the pyenv env, so the environment must be selected *before* `pip install -e .` or the editable install lands in the wrong interpreter. Documented in `CONTRIBUTING.md`'s new **Moving the checkout** section, along with the hooks fix above.
- Removed two stale vendor datasheet PDFs from `doc/`.

experimental/profiling (2026-09-11)

GUI responsiveness. Since the mandatory Kaiser anti-alias filter and oversampled streaming landed, the GUI stuttered on every processing call: bearable at 3 enabled channels, progressively worse to 8, on a 4824A. The bottleneck was not known – USB transfer, DSP and dearpygui rendering were all plausible – so this branch builds per-stage timing first and fixes what the measurements actually indict.

It was not a throughput problem, and that is why it was hard to find. Processing never got ahead of acquisition and the frame queue never backed up. It could not: the 32-frame ring cache is *designed* to skip to the latest frame, so a main-thread overrun surfaces as latency and never as a backlog. Looking at queue depth found nothing because there was nothing there to find.

Three independent causes, each **linear in enabled channel count** – which is why the symptom alone could not separate them.

Added

- `src/rev80/_profile.py` – per-stage timing for the whole pipeline, from the driver callback to `render_dearpygui_frame()`. Thirteen named stages grouped by *thread*, because which thread a cost

lands on is the entire question: main-thread time blocks the mouse, hardware-thread time steals the GIL, and the two need completely different fixes. `gui.render` beside `proc.total` is what makes that call unambiguous from now on. Near-zero when off – measured **442 ns/call disabled** (3015 ns enabled) net of loop baseline, i.e. 0.27 ms per wall-second at the busiest call site, so the instrumentation ships permanently rather than being compiled out. Bounded by construction (fixed-length ring per stage): a diagnostic that becomes the next S-02 is worse than no diagnostic.

- **rev80 --profile** (and `REV80_PROFILE=1`, reachable from a desktop launcher where a flag is not) – logs the stage table on exit, from inside `cleanup()`'s guarded ordering where it can never prevent `ps4000aCloseUnit`.
- **scripts/profile-pipeline** – channel-count sweep printing one stage table per count, simulated or `--hardware`. Its `--raw-rate` **defaults to the hardware-realistic clock, not the nominal one**, because the headline defect below is invisible at exactly 25600 Hz.
- **tests/test_adc_conversion.py** (6 cases x 14 ranges), **tests/test_display_rate.py** (38), and a bit-exactness suite for `_running_median` in **tests/test_peak_selection.py** (110).

Fixed

- **Cause 1 – `decimate_to_rate` designed a 512,821-tap FIR on every call, on real hardware only.** `resample_poly` builds a $2 \cdot 10 \cdot \max(\text{up}, \text{down}) + 1$ tap filter, so the ratio's denominator is a direct cost multiplier. It was nominally bounded, but `limit_denominator` was applied to each rate *separately before dividing* – and both are integers there, so each reduced to denominator 1 and the ratio was never bounded at all.

Invisible offline, because `SimulatedSensor` reports exactly 25600 Hz, which reduces against every display rate to a small integer factor. The driver did not: the streaming interval was requested in whole **microseconds**, and `int(1e6 / 76800)` truncated 13.02 to 13, giving 76923 Hz and a reported

1. 5120 and 25641 are coprime. Measured, 12800-sample block:

raw rate	up / down	taps	ms/call	out length
25600.00 (simulated)	1 / 5	101	0.64	2560
25641.00 (hardware)	5120 / 25641	512 821	73.62	2556

At 8 channels that is 589 ms of main-thread work against a 500 ms frame. **Green CI was not evidence here either** – the whole test suite passed at 0.64 ms while the instrument stalled at 73.6.

It was also a measurement defect, not only a speed one: $2556 < \text{nperseg}$ silently tripped Welch's fallback, so the delivered bin width was not the one the Spectrum tab stated.

Fixed in two independent halves, either of which is sufficient:

- **The streaming interval is now requested in nanoseconds, snapped to the device's clock grid.** This is a frequency-axis accuracy fix in its own right – every displayed line was 0.16% high, so a 1000 Hz line read 1001.6 Hz.

The naive version of this change (`round(1e9/76800) = 13021 ns`) was measured on the 4824A and did *not* do what the arithmetic said. Probing the driver with a range of intervals and reading back what it used shows the reachable points are **12.5 ns apart** – an 80 MHz timebase – and that the driver **floors** to the grid rather than rounding, so 13021 lands a whole grid point high:

requested ns	returned ns	rate/ch Hz	/osr Hz	ppm vs 25600
13000 (and 13012)	13000	76923.08	25641.03	+1603
13021 (naive round)	13012	76852.14	25617.38	+679
13025 (grid-snapped)	13025	76775.43	25591.81	-320

8e7/1041 and 8e7/1042 straddle 76800 and 1042 is the nearer, so -320 ppm is the best this hardware can reach. Rounding to nearest on the grid and then ceil-ing into whole ns – so the driver's floor lands where intended – gets there: **5x better than shipped, not the 100x the ns resolution alone suggested**. Recorded here because the arithmetic and the hardware disagreed, and the hardware won.

None of it is trusted blind. The driver still writes back the interval it really used and that readback is what everything downstream believes, so a device with a different timebase floors to its own grid and reports it, exactly as before. The snap is only ever an optimisation.

- **The ratio bound is applied to the ratio**, via the coarsest denominator cap that lands within `_RESAMPLE_RATE_TOL`. Every shipped preset now reduces to its exact integer factor; an off-preset `F_max` costs a few thousand taps instead of half a million. Measured worst case under the cap is **1.05 ms**, and at the real hardware rate **73.62 ms -> 0.42 ms**, with the output length back to the declared 2560.

These two halves are independent, and it is worth being clear about which one does what: **the ratio bound is what makes it fast** (0.42 ms at either clock), and **the grid snap is what makes it accurate**. Neither substitutes for the other.

The user-facing `F_max` preset is unchanged and stays a round number. The sub-Hz difference between requested and achieved display rate is internal, where it belongs – it is what the frequency axis is correct against – and is never surfaced as a fiddly number on a control. Same treatment `highpass_fc` already gets: a declared edge, with the real value underneath.

Two consequences of a now-fractional raw rate were chased down: the HDF5 and monitor-session readback paths did `int(samplerate)`, which would truncate 25599.67 to 25599 – coprime again – so **replay would have silently taken the expensive path the live display no longer does**, violating the rule that replay reproduces what the live display showed. Both are `float` now, as is what `MonitorWriterThread` writes. `receive_data`'s missing-key fallback also read `config.samplerate` (display) for what is a raw-rate block; unreachable today, wrong if ever hit, now `raw_samplerate`.

- **Cause 2 – `picosdk.functions.adc2mV` is a per-sample Python loop, run inside the driver callback holding the GIL**. It is literally `[(np.int64(x) * vRange) / maxADC.value for x in bufferADC]`, boxing a numpy scalar per sample, and its cost is therefore stolen directly from the GUI thread. Replaced with a vectorised `_adc_to_mv()`:

samples/ch	<code>adc2mV</code>	vectorised	speedup
4 096	4.22 ms	0.021 ms	201x
19 200	19.86 ms	0.052 ms	382x
38 400	40.28 ms	0.112 ms	361x

At ~1.03 us/sample and 76.9 kHz per channel that was 0.079 CPU-seconds per wall-second per channel – **0.634 s/s at 8 channels**. Measured effect on a 60 Hz-style main loop, p95 tick latency against a 0.5 ms target:

channels	1	3	4	8
adc2mV	1.42 ms	4.67 ms	5.75 ms	5.76 ms
vectorised	0.58 ms	0.59 ms	0.59 ms	0.59 ms

The replacement is **bit-identical**, not merely close, over every voltage range across the full int16 domain – provided the operation order is kept: `x * vRange / maxADC`, never `x * (vRange / maxADC)`, which rounds differently in the last bit. `tests/test_adc_conversion.py` asserts the equality *and* that the tempting pre-divided form is not equivalent, so the comment explaining it is backed by a test.

This also explains an earlier workaround: `RAW_SAMPLERATE_HZ` was lowered “to relieve GUI lag while streaming 4 channels” (hotfix/`RAW_SAMPLERATE`, 2026-09-09). Lowering the rate reduced the sample count through this loop. That trade may now be reclaimable.

- **Cause 3 – the per-channel peaks table was destroyed and rebuilt every frame.**

`_update_fft_peaks_table` deleted every column and row and built them again, once per vibration channel per frame, from *both* branches of `_update_freq_plot` – so it ran even with zero peaks.

`select_peaks` reports a corpus median of 40 lines, so that was **~164 widget create/destroy operations per channel per frame** plus a full dearpygui table layout pass: ~1300 per frame at 8 channels, roughly 80% of all per-frame DPG traffic.

Columns and rows are now a persistent pool, created on demand, relabelled or `set_value`’d thereafter, with surplus rows hidden rather than deleted; a frame whose contents are unchanged pushes nothing at all. Every plot *series* in this file was already updated that way – this is the same idea applied to a table. Self-healing if the table is ever rebuilt underneath it, in the spirit of `_ensure_legends`.

Changed

- **`peaks._running_median`’s edge handling is vectorised.** It was a Python loop of `np.median` calls, one per edge bin, and it was **91% of the function’s cost** – the `scipy.ndimage.median_filter` over the interior is only 0.047 ms of it, which is the opposite of where one would look.

	before	after
<code>median_filter</code> (interior)	0.047 ms	0.047 ms
edge bins	1.161 ms	0.408 ms
<code>_running_median</code> total	1.274 ms	0.563 ms

The statistic is unchanged and that is the entire constraint: the truncated window and the measured error table that chose it over zero-padding, reflection and replication are untouched, and the new code is asserted **bit-identical** to the loop it replaces across 110 length x width combinations. Even widths are covered deliberately – unreachable today since `local_noise_floor` forces an odd window, but the edge window is `[i-half, i+half]` inclusive, i.e. `2*half+1` samples whatever the parity, and sizing it by `width` would have quietly shortened every even case.

- **The stale-frame watchdog no longer spawns a thread per frame.** `_schedule_status_timeout` cancelled a `threading.Timer` and constructed a new one on every displayed frame, then flipped a dearpygui widget *from that timer thread*. Both halves were wrong – the second more so than the first, since no DPG call belongs off the render thread. It is now a deadline checked in `_poll_new_frames`, which already runs every tick whether or not a frame arrived, which is exactly when the check needs to happen.
- **Envelope analysis is gated on being on screen**, which its docstring always claimed and the code never did – `envelope_enabled` is a config flag, not a statement about the selected tab. With the tab enabled but the user on Spectrum, a `butter` design, a `sosfiltfilt`, a Hilbert transform and (on an auto band) a

`suggest_band` convolution ran for every channel every frame for a plot nobody could see: ~2.8 ms/channel, plus 1.4-1.9 ms when auto. The gate tests the *plot*, not the tab – an unselected tab still renders its own header button and reports visible either way.

- **`suggest_band` is cached per channel** rather than recomputed every frame. Also better behaviour, not only cheaper: `_on_env_auto_band`'s docstring already says the band should “stay put across frames instead of drifting each time”, and a band that moves every frame makes the envelope plot's own axis unstable. Invalidated when the band fields are edited, when the Envelope tab is toggled, and by the Auto button – which means “pick one from the frame I am looking at now” and must not return an earlier frame's answer.
- Two per-frame `configure_item` calls that push an unchanged value are now change-only: the four browse-button `enabled=` flags (which change twice in a session), and the Frame info card's `height=`, which forces a dearpygui relayout each time.

Where it stands now

`./scripts/profile-pipeline --hardware --channels 1,3,4,8 --seconds 12` on the 4824A (s/n 13290/0013), ms of work per wall-second:

stage	1 ch	3 ch	4 ch	8 ch
<code>usb.poll</code>	57.9	73.7	83.1	128.6
<code>usb.adc2mv</code>	9.2	12.8	14.6	23.0
<code>usb.antialias</code>	7.8	19.8	26.6	58.3
<code>ingest.receive</code>	1.4	2.5	3.1	7.5
<code>proc.total</code>	13.9	30.8	40.2	84.1
<code>proc.decimate</code>	2.2	5.0	6.3	13.3
<code>proc.psd</code>	6.3	13.7	17.9	36.6
<code>proc.peaks</code>	3.5	7.8	10.3	22.1

Zero overflow, zero rate degradation at every count. `usb.poll` contains the three stages below it – the app callback runs synchronously inside `ps4000aGetStreamingLatestValues` – and `proc.total` contains `proc.decimate` / `psd` / `peaks`, so these are nested, not additive; the harness says so in its own output.

`proc.total` at 8 channels is 39.5 ms mean against a 500 ms frame: ~8% of the main thread, where it was over budget before.

The profiling's own next finding, recorded rather than acted on. With `adc2mv` gone, `usb.antialias` is now the largest single cost on the acquisition thread – the mandatory Kaiser FIR decimating a (38400, 8) block per frame. That is real, necessary work rather than a defect, and 58 ms/s is not currently hurting anything. It is simply where the next look should start if one is ever needed, and it is only visible at all because the instrumentation now exists.

Not done, deliberately

Adaptive streaming rate by channel count, and capping the enabled channel count. Both were on the table at the start and both trade away measurement capability to work around a Python loop and an unreduced fraction; Causes 1 and 2 remove the reason for either. Moving `process_samples()` off the render thread is also not done: it would change the `new_frame_event` contract that browse mode, `collect_sample`, the monitor and the tests

all depend on, and Cause 1 alone removes ~589 ms of the ~610 ms main-thread budget at 8 channels. If a hitch survives, it earns its own branch and its own measurements.

build/ci — release automation (2026-09-17)

Tagging `vX.Y.Z` now builds the Windows installer and the Python wheel and attaches both to a **draft** GitHub Release, replacing the manual “boot into Windows, pull, run `scripts/build.sh`” step. Prompted by the move from the self-hosted `catherby` remote to `github.com/cascadia-turbo-works/rev80`.

Not yet executed against a real remote — see *Rehearsing it* in `CONTRIBUTING.md` before trusting a release.

Added

- `.github/workflows/release.yml` — four jobs on a `v*` tag: a test gate, a wheel/sdist build (ubuntu), a driver-less installer build (windows), and `gh release create --draft`. The gate exists so a tag cannot cut a release from a red tree; it runs one Python version, since the full matrix already ran on the branch.
- `scripts/build.sh wheel` — builds the wheel and sdist, which `build.sh` had never done, and the only target that runs off Windows.
- `scripts/build.sh ... nodlls` — skips DLL collection. Hosted Windows runners have no PicoSDK and it has no reliable unattended install, so CI installers are **driver-less**: they work, but the user installs PicoSDK themselves and the `.iss` already warns when it is missing. A local `./scripts/build.sh` is unchanged and still bundles the DLLs.
- **build added to the dev extra.**

Fixed

- `fetch_font.sh` **silently shipped a font-less installer on failure**. It has no `set -e` and returned the status of its final `echo`, so a failed `curl` or a missing `unzip` exited 0; `build.sh` carried on and `rev80.spec` printed its “fonts not found” warning into a log nobody reads. Every failure path now exits non-zero with a reason.
- **...and it downloaded a font the repo already tracks**. `assets/fonts/CommitMonoNerdFont-Regular.otf` is committed and byte-identical to the download (sha256 `4eda301c...`, verified before the change). The tracked copy is now the primary source and the download a fallback, which takes the release build off the network — and off Git Bash’s non-guaranteed `unzip` on the Windows runner.

Changed

- `ci.yml` **triggers on branch pushes only** (`push: branches: ['**']`). A bare `push:` also matches tags, so tagging would have run the full 4-job matrix alongside `release.yml` and its own gate.
- `CLAUDE.md` **no longer claims the pre-commit hook stamps `_version.py`**. It has not since the `setuptools_scm` move; the hook has carried a comment saying so while the doc said the opposite. The release workflow’s correctness rests on the real mechanism, so the passage is now explicit about it.

Notes for the next person

- **fetch-depth: 0 is load-bearing and its failure is silent**. `setuptools_scm` reads `git describe`; a shallow checkout has no tags, falls back to `0.0.0+unknown`, and ships `Rev80Setup-0.0.0+unknown.exe` with nothing failing. Both build jobs assert against that string rather than trusting the checkout. A dirty tree is the same hazard from the other end — it appends `+d<date>`, which is why `_version.py`, `installer/version.iss`, `drivers/*.dll` and the fetched font are all gitignored.
- **python -m build cannot run from the repo root**. This repo’s own `build/` directory shadows the build PyPI package as an implicit namespace package: `import build` succeeds and `python -m`

`build` dies with *No module named `build.__main__`*. Compounding it, `python` is a pyenv shim that picks its version from the cwd, so simply running from elsewhere selects a different interpreter. `build.sh wheel` handles both — resolve the interpreter to an absolute path, then run from a scratch cwd with the repo passed explicitly.

- **A tag trigger ignores branches.** GitHub Actions has no notion of “tagged on main”; any `v*` tag anywhere builds. Accepted deliberately. Legacy `rc0.x` tags do not match `v*`.
- **The wheel is a release asset, not a PyPI package.** `picosdk` is a direct git URL dependency and PyPI rejects those.
- **Releases are drafts** because the exe and installer remain unsigned.
- The `--sdist` and `--wheel` invocations are deliberately separate so the wheel is built from the source tree, on the theory that `setuptools_scm`'s git-tracked file finder would drop the gitignored font. Measured: it does not, `package_data` wins. Kept as belt-and-braces and recorded as measured rather than left as a claim.

[0.1.0] - 2026-09-01

First tagged release. The sections below were written branch-by-branch during development and are kept as originally recorded, grouped here under the release they shipped in.

feature/raw-stream-retention (2026-08-31)

Added

- **Acquisition rate decoupled from the display `F_max`**. Envelope/demodulation analysis needs Nyquist headroom into the 2–20 kHz range where bearing housing resonances live, but `F_max` is a *display* setting users routinely set to 1–2 kHz per ISO route-monitoring convention — and at that `F_max`, acquisition itself discarded everything above Nyquist before any analysis ever saw it. The raw stream is now retained at full bandwidth and `F_max` governs only what the Spectrum tab displays.
 - **`AcquisitionSettings.raw_samplerate / .raw_blocksize`** — a second, independent rate pair, fixed at `RAW_SAMPLERATE_HZ` and *not* maxfreq-derived. This is what `PicoScopeStream` acquires, what `VibeSample` / the HDF5 file / `frame_cache` hold, and what envelope analysis reads. `samplerate / blocksize` keep their previous maxfreq/binsize derivation and become display-only. One frame is one time window at two sample counts: `raw_blocksize` and `blocksize` span the same `acquisition_period`.
 - **`RAW_SAMPLERATE_HZ = 40000`** as shipped here, validated on a PicoScope 4424A up to 4 simultaneous channels. (Later reduced to a briefly-set `10_000` to relieve GUI lag, then corrected to **25600** = 2.56 × the top preset — see hotfix/RAW_SAMPLERATE under [Unreleased], which is also where the display-rate derivation this split exposed was fixed.)
 - **`collector.decimate_to_rate()`** — anti-alias filter + resample from the raw rate down to the display rate before Welch. Generalises `picoscope.antialias_decimate`'s integer-factor decimation to an arbitrary rational ratio via `resample_poly`, because a maxfreq-driven display rate is not in general a clean divisor of a fixed raw rate. `antialias_decimate` itself is deliberately untouched — that is the electrically validated hardware acquisition path — but the two are pinned to the same stopband target (`_AA_STOPBAND_DB`, through `scipy.signal.kaiser_beta`) rather than to two independently chosen filter designs. The ratio is `Fraction.limit_denominator(1000)`, and the function returns the rate it *actually* achieved (`raw_rate * up / down`), not the one it was asked for.
 - **`DataCollector.eu_scaled_raw(ch, sample)`** — the high-pass-filtered signal in the sensor's own native EU at the raw rate. Converts mV → EU but skips the SI/target-unit conversion and every

integration order: envelope analysis demodulates the raw sensor signal directly and has no target unit of its own.

- `DataCollector.current_frame()` — the `{ch: VibeSample}` dict for the frame currently displayed, using the same streaming-vs-browse cursor selection `process_samples()` uses, so a consumer needing the raw `VibeSample` rather than a decimated `ChannelResult` reaches the same frame without re-deriving the index.
- The **Envelope tab is rewired** onto `eu_scaled_raw()` via `current_frame()`, instead of `process_sample()`'s now-decimated `ChannelResult`. Without this the one feature the whole split exists for would still have been display-rate-limited.
- `simulation.RawRateView` — presents `config` at `raw_samplerate / raw_blocksize` to the signal generators, so `SimulatedSensor` generates *and reports* at the same rate `PicoScopeStream` delivers. Without it the simulated path silently generated at the display rate and offline dev/CI never exercised the raw → display decimation at all — the simulator would have defeated the retention it is supposed to test. Proxying leaves every generator, and the tests that call them directly against a real `AcquisitionSettings`, unchanged.
- `scripts/validate-streaming-capacity` — sustained continuous-streaming stress test against a real PicoScope: configurable duration, rate and channel count, reporting overflow count, rate-degradation transitions (the streaming-rate watchdog in `picoscope.py`) and actual vs. requested raw ADC delivery rate. Exit status is 0 only on zero overflow *and* zero degradation transitions. This is how `RAW_SAMPLERATE_HZ` was chosen, and it is the reusable diagnostic for re-validating it against different hardware or a different expected channel count — rather than the measurement being re-derived by hand each time.
- Monitor config dialog: a **>10 GB/year red warning** on the storage estimate. Per-year storage no longer scales down with a low `F_max`, because every stored frame is now the raw-rate capture.

hotfix/cli-ux-refactor (2026-08-30)

Added

- `desktop.py` — **Linux desktop integration**, driven by `rev80 --install-desktop-entry / --uninstall-desktop-entry`. Writes a `~/.local/share/applications/rev80.desktop` entry plus a hicolor PNG icon set under `~/.local/share/icons/`, then best-effort refreshes the desktop and icon caches so the launcher appears without a re-login. Windows is unaffected — the frozen build gets its Start Menu shortcut from the Inno Setup installer — and both commands refuse to run off Linux rather than half-installing.
 - The launcher's `Exec` is resolved as the `rev80` console script **next to the running interpreter** first, falling back to `PATH`. That is where pip places console scripts for a `--user`, `venv` or system install, so the entry points at the environment `rev80` was actually installed into rather than whatever happens to be first on `PATH`. If that directory is not on `PATH`, `install()` says so and prints the `export` line — the launcher works either way, the terminal command does not.
 - **The icon set is PNG, not the source SVG.** Qt/KDE's SVG renderer does not render this icon correctly, breaking it in both the launcher and the taskbar; a plain hicolor PNG set at fixed sizes works everywhere.
 - `uninstall()` removes only the entry and the icons it installed, and reports “nothing to do” rather than failing when there is nothing there.
- **Unified `rev80 CLI`** — a `headless` subcommand, top-level info commands (`--init-config`, `--list-devices`, `--list-sensors`, `--edit-config`) that need neither GUI nor hardware, and `--version`. `rev80-headless` remains as a standalone shortcut.
- **CONTRIBUTING.md** — `README.md` split into user-facing documentation and dev/build concerns (dev environment, project layout, testing, icon regeneration, Windows installer build).

Fixed

- **A non-editable `pip install .` produced a broken app.** `logging.yaml` and the icon font were not shipped as package data, and `resource_path()` assumed a source checkout — it resolved relative to the *project root*, which does not exist once the package is installed normally. `resource_path()` now resolves relative to the package's own directory, correct for an editable checkout, a normal install and a frozen `sys._MEIPASS` bundle alike; anything resolved this way must physically live under `src/rev80/` and be declared in `pyproject.toml`'s `[tool.setuptools.package-data]`. The editable checkout every test runs against is precisely the one layout that hid this.
- **`.githubhooks/pre-commit` restored** — the ruff check and the `_version.py` git-describe stamp, extended to re-render `doc/*.pdf` from `README.md`, `CONTRIBUTING.md`, `doc/PROGRESS.md` and `doc/CHANGELOG.md` via `scripts/render_md.sh` when those are staged. Ruff is warn-only here against 209 pre-existing findings — a gate on the backlog, not on the commit.

Changed

- `_paths.data_dir()` is always `~/Documents/Rev80/data`, in development and frozen builds alike, instead of `./DEVDATA` in development. `DEVDATA` remains test scratch space only, hardcoded independently in `tests/`. A dev run and a shipped run now write measurements to the same place the user is told to look.

Removed

- `ScopeSensor.target_unit` and `effective_target_unit()` — dead. The registry field was never user-settable and always defaulted to a wrong value. The display/integration target is a per-channel setting (`channel_target_units`) and is unchanged: one sensor may be wired to several channels with different targets, so the target never belonged on the sensor definition.

feature/spectral-averaging (2026-08-29)

Added

- **Linear power averaging of the spectrum over N frames**, with an enable checkbox and N in the acquisition dialog (off by default). Welch's method *is* linear power averaging, but since the F-8 fix set `nperseg = blocksize` it runs exactly one segment per frame — so there was no averaging anywhere in the chain, and `welch_overlap` had nothing to act on. Each bin of a single-segment estimate is $\chi^2(2)$, with a standard deviation equal to its own mean, which is why the floor looks rough and a small line is hard to pick out of it.
 - `AcquisitionSettings.averaging_enabled` / `.n_averages` / `.n_averages_effective` (clamped to `cache_frames` — you cannot average more frames than are retained); `ChannelResult.n_averages` reports the count **actually achieved**.
 - A derived **Avg. Window** field reading `16 x 0.5 s = 8 s`, flagged when capped by the cache: N alone is hard to reason about, and how long the machine must stay steady is what the analyst actually needs to know. A frame is exactly `1/binsize` seconds.
 - A live “averaging 12 of 16 frames” line beside the peak count, showing “of N” only when the delivered count falls short.
 - `DataCollector._psd_and_overalls_for()` — the per-frame Welch PSD and 5-order overalls, extracted from `process_sample` so the averaging accumulator reaches earlier frames through exactly that code rather than a second copy. Two divergent copies of one path was the direct cause of M-05.
 - `tests/test_spectral_averaging.py` — 19 tests.

The rule tying live and replay together

The average is the N most recent **valid** frames up to and including the frame being displayed. Live that is the last N received; browsing it is `frames[cursor-N+1 ... cursor]`. One rule, so stepping forward through a loaded file reproduces exactly what the live display showed at that moment — the same replay-fidelity property F-4 established for the high-pass.

Nothing is baked into the file. The HDF5 stores individual raw frames, so a capture taken with averaging *off* can be given 16 averages after loading, and changing N recomputes without reloading. Averaging is a view on stored data, not a property of it.

Decisions, each pinned by a test

- **Power domain, never amplitude.** Average $|X|^2$, then sqrt at the end. On a coherent line the two are identical, which is exactly why this is easy to get wrong and stay green — it shows only on the noise floor, where averaging magnitudes converges about **11% low** (a Rayleigh magnitude has mean $0.886 \cdot \sqrt{\mu}$ against the RMS). The overall combines as `sqrt(mean(squares))` for the same reason.
- **Above the per-frame PSD cache**, so changing N invalidates no per-frame work and does not have to join `psd_key`.
- **Overloaded records are rejected from the average**, including when the displayed frame is the overloaded one — analyzer practice, and the same reasoning F-5 used to keep them out of the trend. The waveform, the scalars and the overflow flag still come from the displayed frame, so nothing is hidden; only the spectral estimate is protected.
- **Crest factor and kurtosis are NOT averaged.** Averaging is for steady-state estimation; those exist to catch the frame that is *not* steady.

*Recorded because it is the point of the whole exercise: the first revert-check pass showed that swapping power averaging for amplitude averaging **passed all 17 tests**, despite the design note calling it the one thing that must be right. Two tests were added asserting against both candidate answers so the wrong one cannot pass.*

Measured

Noise plus a 300 Hz line, $F_{\max} 1000 / df 2$:

N	floor CV	$1/\sqrt{N}$ predicted	floor level	line amplitude
1	0.5260	0.5260	0.03387	0.22366
4	0.2248	0.2630	0.03626	0.22183
16	0.1189	0.1315	0.03740	0.21965
64	0.0604	0.0657	0.03800	0.21878

Scatter tracks $1/\sqrt{N}$; the line is untouched at -2% across a $64\times$ change. The floor *level* rises 12% over that range — not drift, but the Rayleigh-mean-to-RMS convergence the power-domain argument predicts, and the clearest confirmation that the average is being taken correctly.

Still open

`welch_overlap` remains inert: averaging across frames does not overlap segments *within* a frame. Making it real would roughly double the available averages for the same wall time.

round 2: vibration-analysis integrity (2026-08-29)

Second remediation round from the three-discipline audit, addressing the vibration-engineering findings the first round did not cover. Round 1 fixed *how* the numbers were computed; this round fixes *what they were computed over*, and adds the diagnostics an instrument in this class needs.

Added

- **envelope.py** — envelope (demodulation) analysis, the audit's only "blocks stated purpose" gap. `envelope_spectrum()` band-passes around a structural resonance, takes the Hilbert magnitude, removes the DC term and returns a coherent-gain-corrected amplitude spectrum; `suggest_band()` picks a demodulation band from the frame. A bearing defect's impulses ring a housing resonance at 2–20 kHz and are buried under the 1x in the raw spectrum, but appear as a clean line at the defect rate with $\pm 1x$ load-zone sidebands in the envelope — months before the broadband overall moves. Verified end to end against the simulated oracle: auto band 2546–5046 Hz against a true 4000 Hz resonance, BPFO expected 325.8 Hz found at 324.0 Hz (within one bin) at **125× SNR**, with the 1x and lower sideband next. New **Envelope** plot tab with band controls and an Auto button.
- `_dsp.crest_factor()` / `_dsp.kurtosis()` and `ChannelResult.crest_factor` / `.kurtosis` — the two impulsiveness scalars a broadband overall averages away entirely. Non-excess kurtosis (Gaussian = 3.0), which is what condition-monitoring practice quotes. Computed on the band-limited displayed trace, never on the Hann-tapered array the overall uses, whose taper is an amplitude envelope that would corrupt any peak statistic. Trended, persisted to HDF5 and to the monitor session, and shown on each result card.
- **Declared measurement band** — `AcquisitionSettings.band_fmin` / `.band_fmax` with resolved properties, `util.ISO_BAND_PRESETS` (ISO 20816 10–1000 Hz and low-speed 2–1000 Hz), an acquisition-dialog preset combo plus editable edges, `ChannelResult.band_fmin` / `.band_fmax`, HDF5 and monitor-writer persistence, and the band shown on the result card and both trend axes.
- **simulation.GenerateBearingVibration()** — a physically realistic bearing-defect model: impulse train at the defect rate, each impulse ringing a structural resonance, amplitude-modulated at the shaft rate by the load zone, with cumulative slip jitter, and `severity=0` as the healthy negative control. Now the default `SimulatedSensor` source.
- **tests/test_declared_band.py** (46), **test_bearing_oracle.py** (20), **test_diagnostic_scalars.py** (13), **test_envelope.py** (14).

Fixed

- **M-06 — the overall was not band-limited.** It was the RMS of the whole filtered block, so its band was `highpass_fc ... fs/2`, and `fs/2` is $1.28\times$ – $2.56\times$ `maxfreq` depending on where the power-of-two rounding in `samplerate` lands — **2.048× at the 500/1000/2000 Hz presets**. F-9 had already truncated the *spectrum* at `maxfreq`, so the number on the result card and the picture beside it described different bands, and nothing recorded which. Content the user had explicitly excluded via `F_max` still reached the trend (+25% on the audit's case, enough to cross an ISO 20816 zone boundary), and overalls were not comparable across sessions taken at different `F_max` — which silently invalidates long-horizon trending.
 - All five integration orders now share one masked, Hann-tapered path. Order 0 previously skipped the taper, correctly, because a passthrough performs no transform-domain multiply. Band-limiting removes that premise: the mask *is* such a multiply, hence a circular convolution in time.
 - The un-tapered Parseval alternative was measured and rejected: exact in band, but with a rectangular window's -13 dB sidelobes it let a $3\times$ tone at 30 Hz past a 100 Hz edge at only **–22 dB**, inflating the overall +2.70%. Hann rejects it by **–84 dB**, and -100 to -144 dB elsewhere, for $4.9e-4$ worst-case in-band error.

- Hardware (4424A, AWG loopback, band 10–1000 Hz at F_{max} 2000): 800 Hz +0.0 dB, 950 Hz −0.0 dB, 1000 Hz −3.1 dB, **1100 Hz −58.2 dB**, 1300 Hz −58.6 dB, 1600 Hz −58.9 dB — a measured 58 dB cliff at the declared edge.
- **ISO 2954 — the high-pass −3 dB point sat on the declared band edge.** A 4th-order Butterworth designed at 10 Hz reads 29% low at 10 Hz, the very frequency the standard names as the bottom of its declared band. `highpass_fc` is now the declared edge and the knee sits below it at $f_{\text{edge}} * (A^2 / (1-A^2))^{(1/2N)}$; at $N=4$, $A=0.9$ that is $0.8342 \times f_{\text{edge}}$, so a 10 Hz edge designs at 8.34 Hz and reads −0.915 dB at 10 Hz. Safe only because the band mask now removes sub-band energy exactly, so the $1/\omega^2$ blow-up the higher knee implicitly guarded against cannot reach the result. Hardware: 100 Hz +0.00 dB, 20 Hz −0.04 dB, **10 Hz −1.05 dB** (was −3.0), 5 Hz −18.40 dB.
- **M-12 — anti-alias stopband was ~55 dB against an ~80 dB expectation.** `scipy.signal.decimate(ftype='fir')` uses a Hamming kernel; measured **−60.0 dB** worst case, capping usable dynamic range regardless of the ADC resolution negotiated elsewhere. Replaced with a cached Kaiser design (100 dB, 0.20 transition, 259 taps at $q=4$): **−111.7 dB**, passband flat to $1e-4$ at F_{max} . The wider 0.25 transition saves 52 taps but starts eating the passband at F_{max} . The longer kernel costs nothing at block edges — measured on one block against the analytic RMS, hamming +0.2416% vs kaiser −0.0001%, because the overall's Hann taper already de-weights the edges where the start-up transient lives. Hardware A/B on identical captured samples, $q=8$: **+10.8 dB** at 2600 Hz and +10.3 dB at 3100 Hz where alias leakage rises above the capture noise floor; below it both kernels sit under the floor.
- **M-14 — the simulated bearing signal was not a valid oracle.** Ten pure cosines plus white noise, kurtosis ~3, no impulsiveness, no resonance carrier, no sidebands, no slip. Every diagnostic added this round keys on exactly those properties, so it could not have validated any of them: a broken envelope analyser and a correct one both return “nothing here” on ten pure cosines. Also fixed the operator-precedence bug — `int(freqs[-1] // bearing_multiple*running_rate)` binds left to right, giving ~5000× too many iterations with every out-of-range harmonic collapsing onto the last bin via `argmin`; extracted as `bearing_harmonic_count()` so the two loops needing this arithmetic cannot disagree again.
 - Two constants set by measurement, not from the textbook. **resonance_q**: what governs impulsiveness is ring-down time over impulse period, and at the textbook $Q=40$ the ring-down is *longer* than the gap between impulses, so kurtosis reads 3.08 against a healthy 3.09 — the fault is undetectable. $Q=8$ gives τ/period 0.21 and separates 5.28 from 3.09. **Harmonic phase**: a single shared phase made healthy kurtosis swing 2.03–4.00 across seeds on nothing but how the cosines lined up, overlapping the faulted range; independent phases are both more physical and stable.
 - The healthy control is correctly *sub-Gaussian* (1.54–3.04), not Gaussian: a machine dominated by running-speed harmonics genuinely has kurtosis below 3.
- **S-12 — SimulatedSensor died silently above two channels.** `simulated()` ships a 2-element `scale` while the generator emits one column per enabled channel, so three channels raised a broadcast `ValueError` inside `_stream`, which had no `try/except`: the thread died while `_running` stayed set and `is_streaming` reported a healthy stream producing nothing, indefinitely. The scale is now fitted to the data, `_stream` is guarded and clears `_running` in a `finally`, and `_sample()` emits exactly one column per enabled channel (the old `max(..., N_CHANNELS)` floor gave single-channel configs a phantom second channel).
- **H-08 — AcquisitionSettings.copy()** copied only `maxfreq` and `binsize`, silently dropping every other field including all five per-channel dicts. It now round-trips through `to_dict` / `from_dict`, so a field added there cannot be forgotten here.

Changed

- **The spectral anomaly hook is unwired from the GUI panel** (`GUI_ANOMALY_HOOK_TYPES = ('rms',)`). It fires on essentially every healthy frame: it triggers on `np.any(|spec - baseline| / baseline > threshold)` across every bin while Welch runs a single segment in every shipped preset, so each noise-floor bin is $\chi^2(2)$ with a standard deviation equal to its own mean and $P(\text{some bin of } \sim 2000 \text{ exceeds } 1.5\times)$ is

~1.0 — `consecutive_n` is a delay, not a defence. Bins 0 and 1 are hard-zeroed for integration, so on any velocity/displacement channel they deviate by ~1e12 and fire permanently. The hook, the config schema and the headless front end are untouched, and a stored `spectral / both` is preserved rather than rewritten when the dialog is merely opened. Tracked for a fix-or-remove decision as **R39**.

- Stream pacing moved out of `GenerateBearingVibration_TemporalMethod` — a signal generator should not contain a `time.sleep` — into `SimulatedSensor._stream`, which paces against a deadline so the block rate stays honest when generation is slow.
- `doc/PROGRESS.md` gains **R34–R38** (standards conformance), **R39** and **R40**. R40 records that IEPE bias monitoring is **not achievable on this hardware**: the coupler has a DC blocking capacitor so the bias never reaches the scope on either coupling setting, and the 4000A ranges only to ±20 V against a 24 V supply, so even a direct pre-cap tap would over-range.

hotfix/claude-ultrareview (2026-08-29)

Integration branch for the three-discipline audit remediation. Sections for the individual branches follow below.

Changed

- `util.py` / `gui.py` — the **spectral anomaly hook is unwired from the GUI panel**.
`GUI_ANOMALY_HOOK_TYPES` now restricts the monitor dialog's hook combo to `('rms',)`;
`SpectralThresholdHook`, the config schema and the headless front end are untouched, so reviving it is a one-line change
 - It fires on essentially every healthy frame and has never been useful in practice. It triggers on `np.any(|spec - baseline| / baseline > threshold)` across every bin in the band, while Welch runs a single segment in every shipped preset (`nperseg == blocksize`) — so each noise-floor bin is $\chi^2(2)$, with a standard deviation equal to its own mean. P(some bin of ~2000 exceeds 1.5×) is ~1.0 on healthy data, which makes `consecutive_n` a delay rather than a defence. Bins 0 and 1 are additionally hard-zeroed for integration, so on any velocity/displacement channel they deviate by ~1e12 and fire permanently
 - A stored `spectral / both` — which headless still writes — is clamped for display only.
`GUI._hook_type_to_save()` preserves the original, so merely opening the monitor dialog cannot silently rewrite a headless user's configuration; that is the S-07 failure mode this branch had already fixed once
 - Tracked for a fix-or-remove decision as **R39**. A real fix needs band RMS rather than per-bin, a threshold in σ rather than fixed %, and a default band — i.e. it depends on R34
- `doc/PROGRESS.md` — added **R39** (spectral hook fix-or-remove) and **R40** (sensor-fault detection; records that IEPE bias monitoring is impossible on this hardware — the coupler has a DC blocking capacitor so the bias never reaches the scope, and the 4000A ranges only to ±20 V against a 24 V supply)

feature/peak-selection (2026-08-29)

Added

- `src/rev80/peaks.py` — significance-based spectral peak selection, replacing “report the top N local maxima by absolute amplitude”
 - The local noise floor is not flat: across the 60 channel-spectra of the `old castle` corpus it varies by up to 7× *within a single spectrum*. On `blower 4 - bearing DE.h5 ch2` the median local floor is 9.0 mV overall but 39.4 mV over 1890–2000 Hz, so ranking by absolute amplitude ranked by *how loud the neighbourhood is* rather than by *how far a line stands out of it* — 6 of that spectrum's top 12 came from ripple in the noisy top of the band

- The diagnostic cost was real: 1034 and 1088 Hz are sidebands at $\pm 25/29$ Hz around the 1059 Hz carrier — the bearing-fault signature — and ranked 10th and 4th, so at the shipped display count of 6 the sideband family was broken up and pushed off the table. The instrument was hiding the fault evidence behind noise ripple
- `local_noise_floor()` estimates the floor per bin with a running median, then `select_peaks()` passes **array-valued** height and prominence to a single `find_peaks` call, so admission is evaluated bin by bin against each line's own neighbourhood. Everything that passes is reported; the count becomes an output and the user knob becomes a significance threshold in dB
- **Amplitude reporting is unchanged** — the reported value is still the amplitude of the maximum bin, with no frequency interpolation and no energy summation across a peak. Ranking is still by descending amplitude: significance decides *whether* a line is reported, amplitude decides where it sits in the table
- **tests/test_peak_selection.py** — 626 lines, including an opt-in real-corpus regression suite (`REV80_CORPUS_DIR`, skipped by default since `DEVDATA/` is gitignored)

Changed

- **collector.py** — `process_sample()` step 6 calls `rev80.peaks.select_peaks` instead of `find_peaks(..., distance=5)` + top-N sort. `n_segments` is derived from the Welch parameters rather than assumed to be 1, so the median-to-mean floor correction stays right if `nperseg` ever stops equalling `blocksize`
- **sample.py** — `AcquisitionSettings.peak_threshold_db` (default 9.5), persisted in `to_dict / from_dict`
- **gui.py** — the “Peak Display” count spinner is replaced by a “Peak Sig., dB” threshold. The count still exists but only as a clutter cap on the table and plot markers (default 50, against a measured corpus median of 40 and p90 of 49), and a text line reports how many lines passed and whether the cap is hiding any — without it, a capped table is indistinguishable from a spectrum that genuinely had few significant lines

Measured

- **Default threshold 9.5 dB is set by a false-alarm cliff, not taste.** On pure noise (2001 bins, no lines at all) the mean reported count is 37.9 at 6.0 dB, **0.8 at 9.5 dB**, 0.0 at 12.0 dB
- **Floor window width 65 bins** tuned against the corpus, not argued. Scoring single-frame estimators against a 16-frame-averaged reference floor over 20 files $\times$ 3 channels $\times$ 4 frames gives a broad flat basin from 31 to 65 bins (median |error| 1.097 dB at 31, 1.039 at 45, 1.085 at 65, 1.326 at 129). A *synthetic* spectrum with a deliberately smooth floor prefers 129 — the disagreement is the point
- **Edge handling: truncated window, not the edge replication first proposed.** Replication copies one random bin 32 times, and 32 copies of a single exponential draw dominate a 65-sample median: median |error| 1.480 dB / p95 5.246 dB, against 0.591 / 1.829 dB truncated. Zero-padding (`scipy.signal.medfilt`) is worst, at -3.568 dB mean bias
- Corpus-wide at the default: count median 40, p10 22, p90 49. Under the old top-10 rule 6.67% of reported peaks stood less than $2\times$ above their local floor (18.17% with a flat-top window); the gate admits none
- `wlen` is mandatory with `prominence` — unbounded, a single broad hump spanning the band has prominence 34.6 and its apex looks like the most prominent thing in the spectrum
- Known cost, asserted by a test so it cannot drift silently: **broad features are rejected on purpose.** A 40-bin- σ hump's apex has prominence 7 against a requirement of 144 and never reaches the table. That is right for a line table and wrong if you want broadband resonances flagged; they remain visible in the plot only

fix/measurement-validity (2026-08-28)

Nine measurement-validity defects (F-1 ... F-9) from the vibration-engineering audit, plus one adjacent logging defect (S-09).

Root cause of the whole class: the test suite was degenerate. `tests/test_sample.py` sets `F_max=10000`, `binsize=2` → `fs=32768`, `N=16384`, so bin spacing is exactly 2.000 Hz — then swept tones at 500, 1000, 1500 ... 9500 Hz, **every one an exact multiple of 2**. That is the single case where the block is genuinely periodic in `N` samples, the FFT's circular-wrap discontinuity vanishes, and the integration error is identically zero. `_make_dc` also defaulted `highpass_enabled=False`, so the filtered path was never exercised by any amplitude assertion, and the displacement tests asserted only on the spectrum peak, never on `result.overall`. Phase 1 of this branch was therefore a deliberately **red** commit: 110 tests, 92 failing, before any fix was written.

The same mistake was then made again, and caught: every tone in the new suite was generated as `sin(2πft)`, so **every record started at phase 0** and `x[0]` equalled the DC level exactly — degenerate in precisely the same way bin-centred frequencies are. Real hardware exposed it. `tone()` now takes a `phase` argument defaulting to 0.7 rad.

Added

- `src/rev80/_dsp.py` — windowing helpers for frequency-domain integration, carrying the measured error tables that justify each choice
- `sample.py` — `AcquisitionSettings.nperseg` and `.binsize_actual`: the Welch segment length and the bin width actually delivered
- `collector.py` — `filter_block()`, `filtered_data_for()`, `reset_filter_state()`, `_seed_zi()`: persistent per-channel high-pass state
- `gui.py` — `derive_acquisition_preview()`, a single source of truth for the dialog preview
- `monitor/anomaly.py` — `valid_results()`, a shared validity filter for anomaly hooks
- `tests/test_measurement_validity.py` — 123 tests covering F-1 ... F-9 and S-09, all off-bin, with the high-pass enabled, across block boundaries and at every preset

Changed

- `util.py` — `MAXFREQ_PRESETS` is `[2e2, 5e2, 1e3, 2e3, 5e3, 1e4]`; **the 20 kHz and 50 kHz presets are removed** and `F_max` now tops out at 10 kHz (see F-3)
- `sample.py` — `VibeSample.samplerate / ChannelResult.samplerate` / the HDF5 `samplerate` attribute are now `float`. The true rate is generally not an integer
- `sample.py` — `n_fft_bins` counts **displayed** lines (DC...`F_max`), not the full one-sided transform: 1001 at the default preset, not 2049
- `gui.py` — the spectrum info panel reports `binsize_actual`, and is labelled `F_max` rather than `AA`
- `monitor/writer.py` — no longer carries its own divergent copy of `_write_channel_group`
- CRLF → LF in `picoscope.py` and two `examples/` scripts

Fixed

- **F-1** `collector.py` — **FFT wrap leakage corrupted every integrated overall and waveform.** `rfft` was taken on the raw, un-windowed block and multiplied by $(j\omega)^n$. The DFT treats the record as periodic; unless it is exactly periodic in `N` samples there is a step discontinuity at the wrap point whose spectrum is broadband and low-frequency-weighted, and `n < 0` amplifies it by $1/\omega^{|n|}$. The in-code claim that the round-trip was lossless held only for `n_ord == 0`, and zeroing bins 0 and 1 left the leakage

in bins 2, 3, 4 ... The spectrum was never affected (Welch already windows); what was wrong was the Overall card, the trend, `overall_json`, the anomaly detector's input, and the displayed waveform

- Two treatments, because the consumers want different things. **Scalar overalls:** Hann taper, RMS divided by the window power gain $\sqrt{\text{mean}(w^2)}$. **Displayed waveform:** cannot be tapered — the envelope would be plainly visible on the trace — so overlap-save instead, with a Tukey $\alpha=0.5$ window killing the wrap discontinuity and only the flat middle, where the window is exactly 1.0, returned. `time_vec` is truncated to match
- **Cost, deliberate and documented:** integrated/differentiated traces span the middle 50% of the block
- Measured (1.0 unit 0-pk sine, $f_s=32768$, $N=16384$) — velocity overall at 61.0 Hz **+12.25%** → **+0.05%**; displacement overall at 61.0 Hz **+470.59%** → **+0.18%**, at 120.7 Hz **+777.84%** → **+0.05%**, at 501.0 Hz **+4473.76%** → **+0.00%**; on-bin 500.0 Hz was **+0.00%** before and after, which is exactly why the old suite never saw any of it. Waveform peak, displacement at 501.0 Hz: **+10176.68%** → **+0.24%**
- Taper-width sweep at the worst case (61 Hz displacement): α 0.05 → **+335.84%**, 0.10 → **+7.00%**, 0.30 → **+4.22%**, **0.50** → **+1.47%**. Reflection padding was tried and is far worse (**+1918%**) — it doubles the effective near-DC content. Differentiation ($n=+1$) verified unaffected, worst **+0.018%**
- **F-2 `picoscope.py` — the reported sample rate was not the rate actually used.** `_start_streaming` computed `actual_raw_fs` correctly from the interval the driver writes back, then discarded it (`self._actual_samplerate = self.config.samplerate`); the comment had conflated “hide the oversampling ratio” with “hide the actual rate”. The driver quantises the sample interval to whole microseconds, so at $F_{\text{max}}=2000$ the hardware runs 8333.25 Hz while the app labelled it 8192 — **-1.70%**, rising to **-8.25%** at the (now removed) 50 kHz preset
 - **Confirmed on real hardware** (PicoScope 4424A, AWG loopback on channel A), same captured samples under two labellings: a commanded 1000.00 Hz tone read 983.062 Hz (**-1.694%**) and now reads 1000.012 Hz (**+0.001%**). Mean $|\text{error}|$ across four tones **1.676%** → **0.034%**. Reported as a float — rounding would reintroduce a smaller version of the same error
- **F-3 `picoscope.py` — no anti-alias filtering at all above $F_{\text{max}} = 20$ kHz.** `antialias_decimate()` is a no-op at factor 1, and `max(1, int(min(OSR_TARGET, CEILING / samplerate)))` truncated toward zero: 1.526 → 1 at 20 kHz, 0.763 → 0 → clamped to 1 at 50 kHz. The 50 kHz preset additionally requested 131072 Hz raw, **31% above** the measured `STREAMING_CEILING_HZ` — the exact condition the module docstring says makes the driver silently drop most samples while still reporting `status='OKAY'`
 - A general-purpose IEPE accelerometer has a mounted resonance at 25–80 kHz with 20–30 dB of gain, so at $F_{\text{max}}=20$ kHz an unfiltered 50 kHz component folds to 15536 Hz — inside the displayed band, indistinguishable from real signal, and *larger* than the real signal because of the resonance gain
 - `_choose_osr()` now uses explicit `math.floor` and requires `osr >= 2`. Only $F_{\text{max}} \leq 10$ kHz satisfies $f_s * 2 \leq 100$ kHz, hence the preset removal above. Override by raising `STREAMING_CEILING_HZ`, which requires re-measuring the safe continuous streaming rate on the target hardware and channel count. A maxfreq outside the presets still degrades rather than failing, but now logs a WARNING naming the frequency above which content will alias
- **F-4 `collector.py` — high-pass filter state was reset to zero on every block.** `sosfilt` was called with no `zi`, reintroducing a startup transient at the head of every frame of a continuous stream. Measured over blocks 1–3 of a 200 Hz tone at a 10 Hz high-pass: acceleration waveform peak **+9.41%** → **-0.000%**, displacement overall **+19.15%** → **+0.017%**; with 1000 mV of residual DC offset, acceleration overall **+14321.74%** → **-0.000%**
 - **Two regimes, handled distinctly, because replay is not a stream.** Live streaming filters once per frame in `receive_data`, in order, carrying state, and caches the result on the `VibeSample` since `process_sample` is called repeatedly on the same frame. Replay/browse re-processes stored

- frames out of order and carries no state — verified bit-identical forward and reverse to 12 significant figures. Block 0 of a stream still shows genuine settling; there is no history to carry
- **F-4 follow-up, found by hardware:** `sosfilt_zi(sos) * x[0]` is scipy's documented idiom and is correct when the first sample represents the baseline — true for a step, false for anything oscillatory. On four consecutive real captures the block mean was $-0.06 \dots -0.21$ mV (the true DC) while `x[0]` ranged over **36...305 mV**, and under $1/\omega^2$ that spurious step dominated: worst-block displacement error **+4297.20%** against a settled reference. Now seeded from the block mean (-2.22%). A warm-up pass — filter the block, reuse its final state as its initial state — was measured and is *worse* ($+61.10\%$): it imposes a periodic assumption the block does not satisfy. The branch's own synthetic tests missed this because they used phase-0 sines, reproducing the suite's own central blind spot
 - **F-5 collector.py, monitor/writer.py, monitor/anomaly.py — overload and degraded frames were trended, alarmed on, and stripped on save.** Five gaps forming the classic spurious-alarm mechanism: a clipped waveform reads high with harmonic distortion, the trend records a step change that never happened, the detector fires — and on reload the record looks clean
 1. The overflow bitmask was read from whichever callback completed a block, so a callback raising overflow mid-accumulation had its flag discarded. Now latched with `|=` and cleared on emit
 2. `update_trend` ran regardless of the flags. Flagged frames are now excluded from the trend, while still being displayed and still flagged
 3. `_read_frame_group` hard-coded `overflow=False` and never set `degraded`, so the flags `_write_channel_group` had faithfully stored were never read back
 4. `monitor/writer.py` had a second, divergent `_write_channel_group` that wrote no validity flags at all — the divergence was the defect; they are now one function
 5. The anomaly hooks never read either flag. `valid_results()` is applied to both event evaluation and baseline adaptation: letting a clipped frame into an EWMA baseline poisons the reference for ~ 33 frames just as surely as firing on it
 - **F-6 gui.py — the acquisition dialog computed the sample rate with 2x, not 2.56x.** It duplicated the derivation and used the bare Nyquist minimum, so at the default `F_max=2000` it advertised 4.1 kS/s, 2049 lines, 1.000 s and half the true memory while the instrument ran at 8.2 kS/s, 4097 lines, 0.500 s. Now delegates to `AcquisitionSettings`; verified equal across the full preset grid
 - **F-7 collector.py — the PSD cache key omitted binsize and samplerate.** Switching 2 Hz $\rightarrow$ 0.5 Hz bins returned the identical cached 2049-point, 2 Hz spectrum: the user believed they had quadrupled the resolution and nothing had changed. Masked while streaming, since each new `VibeSample` starts with `psd_mv=None`, so it bit in browse/offline mode and after loading a file
 - **F-8 sample.py — stated line count and bin width did not match the computed spectrum.** `n_fft_bins` returned `blocksize//2 + 1` while Welch used `nfft = int(samplerate/binsize)` and `blocksize = nextpow2(samplerate/binsize) >= nfft`. **40 of 72** preset combinations were wrong; worst `F_max=200/df=20`, a real resolution of 20.480 Hz ($+2.40\%$) and 17 lines claimed against an actual 13. Fixed with `nperseg = blocksize`; re-verified **0 of 72** mismatch. The existing amplitude suite was unaffected — at its `F_max=10000/df=2` the old and new `nperseg` were already both 16384
 - **F-9 gui.py, collector.py — the spectrum was displayed out to fs/2, where alias rejection is ~ 12 dB.** The axis, the peaks table and `find_peaks` all ran to `fs/2 = 1.28 \times F_max` — the anti-alias filter's transition band, -21.8 dB at the folding frequency and effectively 0 dB at `fs/2` itself. The whole point of the `F_max = fs/2.56` convention is that the guard band is never shown. Now truncated at `F_max` before peak-finding. The info label had also called `fs/2` the "AA" frequency, which read as a specification the instrument does not meet
 - **S-09 picoscope.py — the ADC overflow warning inhibit was inverted, flooding the log.** `elif ch in self._overflow_warned: remove(ch)` fired precisely when a channel was *still* clipping, re-arming the warning every other callback — measured **10 warnings from 20 callbacks**. At a 1 ms poll interval that rolls every other diagnostic out of the rotating log during exactly the run being diagnosed. The

clear-down loop also had to move out of `if overflow:`, since a return to zero is the only way to observe it

chore/config-consistency-ci (2026-08-28)

Added

- `.github/workflows/ci.yml` — first CI for the project. There was no `.github/` at all: 300+ tests and nothing ran them.
 - `push + pull_request`, `ubuntu-latest`, matrix over Python 3.10–3.13 (the range the new `requires-python` floor admits)
 - `checkout` → `setup-python` → `pip install -e ".[dev]"` → `ruff check src/ tests/` → `pytest tests/ -q`; `fail-fast: false`, concurrency group cancels superseded runs
 - installs `libx11-6`: `dearpygui`'s `_dearpygui.so` links against `libX11` at load time (confirmed with `ldd`). No GL and no X server are needed because the suite never calls `create_viewport()`.
 - the native PicoSDK driver is deliberately **not** installed; the suite runs against `SimulatedSensor`
 - **Verified end to end locally**, not assumed: a clean Python 3.12 venv built exactly as CI does, with the PicoSDK driver stubbed out and `DISPLAY / WAYLAND_DISPLAY` unset, gives 300 passed / 10 skipped. The `picosdk` `git+https` pin resolved and built a wheel with no special handling, so `--no-deps` is not needed. `dearpygui==2.0.0` publishes manylinux wheels for `cp310–cp313` (checked against the PyPI JSON API), so the whole matrix is covered.
- `util.py` — `amplitude_scale(mode)`, `nearest_interval_preset(seconds)`, `canonical_hook_type(value)`, `hook_type_label(value)`, `ANOMALY_HOOK_LABELS`, `DEFAULT_AMPLITUDE_MODE`, `DEFAULT_ANOMALY_HOOK_TYPE`, `DEFAULT_RMS_ALPHA`, `DEFAULT_SPEC_ALPHA`, and an explicit `__all__`
- `util.py` — `600: '10 min'` added to `MONITOR_INTERVAL_PRESETS`
- `README.md` — Contributing section now documents the one-time `git config core.hooksPath .githubhooks` install step
- **Tests** — 7 new files, 135 tests: `test_gui_save_config.py`, `test_icons.py`, `test_monitor_pretrigger_scaling.py`, `test_sensor_library_integrity.py`, `test_config_contract.py`, `test_anomaly_hook_build.py`, `test_util_small_defects.py`

Fixed

- `.githubhooks/pre-commit` — still ran `ruff check vibechecker/` and wrote `vibechecker/_version.py` after the rebrand moved the package to `src/rev80/`. `ruff` exited non-zero against a nonexistent directory, so the hook blocked every commit for anyone who had installed it. Repointed at `src/ tests/` and `src/rev80/_version.py`.
 - `src/rev80/_version.py` regenerated: it read `rc0.4-10-g11a72f4` while `git describe` gives `rc0.5-44-g7bf4712`. **Shipped builds reported a version 34 commits behind, so a field bug report could not be tied to a build.**
 - Root cause of the silent rot: the hook is not installed by default (`core.hooksPath` is the stock `.git/hooks`), now documented in the README.
- `pyproject.toml` — four defects, each breaking a clean install:
 - `requires-python` was `>=3.8`, two minor versions below the real floor. No module uses `from __future__ import annotations`, so every annotation is evaluated at import: `PEP 604 dict | None` in `sensor.py:46` needs 3.10; `dict[str, Any]` in `config.py:219` and `argparse.BooleanOptionalAction` in `__main__.py:17` need 3.9. `pip` installed happily on 3.8/3.9 and the app raised `TypeError` on first import. Raised to `>=3.10`. Swept for 3.11+/3.12+

- constructs (tomllib, StrEnum, ExceptionGroup, except*, TaskGroup, typing.Self, typing.override, itertools.batched, datetime.UTC, PEP 695 generics) — none present, so 3.10 is correct, not merely safe.
- dearpygui was an optional [gui] extra, but gui.py:6 and icons.py:3 import it at module scope and the rev80 console script reaches both via __main__:main. pip install -e . — exactly what CLAUDE.md instructs — produced a rev80 command that ImportError. Moved into required dependencies; [gui] kept as an empty alias.
 - pandas and matplotlib were declared runtime dependencies but imported nowhere under src/. matplotlib was already in the PyInstaller excludes, confirming it was never needed at runtime. Removed. (Both are still used by the standalone examples/TMS_Digital_Audio.py and scripts/advanced_plots.py, which sit outside the installed package.)
 - numpy, scipy, h5py, pyyaml, plyer, pywin32 were entirely unconstrained, so a shipped installer's contents depended on what PyPI served that day. Added lower bounds (not pins) chosen as the oldest releases that support 3.10 and carry the APIs actually used.
- **pyproject.toml / CI reproducibility** — [tool.ruff.lint] set only ignore, never select, so ruff linted with whatever its *current default* happened to be — and that default moves between releases. On an identical tree: **ruff 0.15.10 → 0 errors, ruff 0.16.5 → 167 errors**, none from a code change. With ruff unbounded in [dev], CI would have gone red on an untouched tree at the next ruff release. Fixed at both layers: select = ["E4", "E7", "E9", "F"] makes the rule set explicit, and ruff>=0.15,<0.17 bounds the version. Both versions now report clean.
 - **gui.py** — _on_sb_save_config called h5py.File(...) but gui.py never imported h5py at module scope; the three other users each did a function-local import and this one did not. The resulting NameError was caught by a broad except Exception and logged as "failed to patch {session_h5}", so the session browser's **"Save Config" button was dead code** and the message pointed the user at disk permissions. Added the module-scope import, removed the three redundant local ones, and narrowed the except to (OSError, KeyError).
 - **icons.py / gui.py** — test_gui_build failed on any clone that had not run scripts/build.sh, with an opaque SystemError: <built-in function pop_container_stack> returned a result with an exception set. assets/fonts/ is gitignored and populated by build.sh, so the font is absent on a fresh clone and on every CI runner; icons.load() passed the nonexistent path to dpq.font(), and the failure inside the context manager surfaced from pop_container_stack naming neither the font nor the path — taking down all of _create_gui(). Now checks for the file and falls back to the DPG default with a warning. *This was the pre-existing failure standing between the suite and green, and the blocker for CI.*
 - **picoscope.py** — the PicoSDK driver (libps4000a) is a separate native install from the picosdk Python wrapper, and picosdk/ps4000a.py instantiates Ps4000alib() at import time, raising CannotFindPicoSDKError when the driver is absent. picoscope.py imported it unguarded, so import rev80.picoscope was fatal without the driver. Reproduced by stubbing find_library: the **whole suite aborts** with 3 collection errors, not 3 modules' worth of skips — note test_antialias.py is pure DSP and was collateral damage. Also contradicted CLAUDE.md's claim of "offline development and CI without hardware". Now degrades to PICOSDK_AVAILABLE = False, mirroring the guard __init__.py:9-17 already uses for this exact import; FindPicoScope() returns [] with a warning. Verified identical results with the driver present and absent.
 - **monitor/controller.py:258** — read the sensor sensitivity under sensitivity_mv_per_eu, a key that exists nowhere in the codebase; session.sensor_snapshot holds ScopeSensor.to_dict() output, which emits sensitivity. The 1.0 default therefore **always** won: the mV → EU division never happened and every burst's pre-trigger trend points came out a factor of sensitivity too large (~10x for a 10.2 mV/

g sensor) against the post-trigger points on the same continuous plot. `engineering_units` on the adjacent line used the correct key, so the unit *label* converted while the magnitude did not — worse than an obvious break. Now rehydrates through `ScopeSensor.from_dict()` once per call so the field name can only be wrong in one place; misleading comment at :246 corrected.

- **README.md:920 / CLAUDE.md:127** — documented the same wrong `sensitivity_mv_per_eu` key. Following the README raised `KeyError` in `from_dict`, and the very next README paragraph correctly said the code divides by `sensitivity` — the two lines contradicted each other. Both fixed.
- **Sensor library could be silently and permanently destroyed.** `ScopeSensorRegistry._load_user` swallowed any parse failure with `except Exception: return []`, and `_save_user` writes whatever `_load_user` returned straight back over the file. `add()` and `delete()` both follow that load-then-save path, so **one unreadable entry erased every calibrated sensor definition, with nothing logged**. Two independent triggers reached it: the wrong documented key above, and `config._atomic_yaml_write` using `yaml.dump` with the **unsafe default Dumper** while every reader uses `yaml.safe_load` — loading a colleague's `.h5` auto-registers its sensors with no prompt and no type coercion, so a non-scalar attribute serialised as a `!!python/object/apply:` tag that `safe_load` then refused. Fixed at all three layers:
 1. `config.py:159` → `yaml.safe_dump`; unrepresentable values now fail loudly at write time and the existing file survives. Also closes a latent escalation: a writer emitting object tags means any future switch to `yaml.load` becomes arbitrary code execution from a shared measurement file. *No RCE today*.
 2. `scope_sensor.py` → `from_dict` coerces every field and rejects containers and arbitrary objects, so junk cannot reach the writer.
 3. `scope_sensor_registry.py` → a bad *entry* is logged with file, index and content and skipped; a bad *file* raises, so a read failure can never become an overwrite. Read-only callers go through `all()`, which degrades to `[]` and logs, so a corrupt file cannot crash GUI construction.
- **config.py / gui.py / headless.py** — two monitor defaults the GUI could not represent and therefore silently rewrote:
 - `hook_type`: `config.py:83` seeds `'rms'` (lowercase); the GUI combo items are capitalised and both `_on_anom_config_change` and `_build_anomaly_hook` compared raw strings, while `headless.py:159` did `.lower()`. On a fresh install `'rms'` matched neither `('RMS', 'Both')` nor `('Spectral', 'Both')`: opening Config → Monitor once **hid both hook groups, built zero anomaly hooks, and silently disabled anomaly detection** behind an Enable switch that was on. Now canonical lowercase everywhere, capitalisation demoted to a display label.
 - `interval_s`: `config.py:74` seeds `600`, which was not a preset member, so the widget showed “1 h” and saving wrote `3600.0` back — **silently changing a 10-minute logging interval to hourly, discarding 5 of every 6 measurements**. Fixed at both ends: `600` is now a preset, and the fallback picks the *nearest* preset rather than a hardcoded 3600.
- **headless.py / gui.py** — `_build_anomaly_hook` bound `period` only inside the `if hook_type in ("rms", "both")` branch but read it in the spectral branch, so a **Spectral-only config raised UnboundLocalError**. In headless this fires *after* `collector.start_stream()`, killing the process with the PicoScope still streaming and never closed — the worst outcome for an unattended run. The two copies fail differently, which is why it survived: `gui.py` reads `period` unconditionally and always raises, while `headless.py`'s `if "spec_ewma_time" in anom_cfg and period > 0` short-circuits, so it only raises when that key is present — which is exactly what the GUI writes. Hoisted in both.
- **Reconciled the four ways the two `_build_anomaly_hook` copies had drifted**: hook-type casing (above); warmup default 30 (GUI) vs 10 (headless) → 10, matching `config.py`'s seed, since at 30 the GUI needed a 3x longer baseline warm-up during which nothing could fire; `spec_n` default 3 (GUI) vs 10 (headless) → 10, since at 3 the GUI fired on a third of the evidence headless required; and the EWMA-

alpha fallbacks, previously hardcoded separately in each copy, now `util.DEFAULT_RMS_ALPHA / DEFAULT_SPEC_ALPHA`.

- **`_pico_loader.py:24`** — computed `Path(__file__).parent.parent / 'drivers' → src/drivers`, which does not exist; `_paths.py:25` already gets this right with three `.parent` s. `ensure_pico_dlls_loadable()` silently returned `False` in development, bundled DLLs were never registered, and picosdk fell back to walking `%PATH%`. Frozen builds use the `sys._MEIPASS` branch and were unaffected. Now delegates to `_paths.resource_path('drivers')` — the duplication was the bug — and logs a warning on Windows when it returns `False`.
- **`collector.py:1187, 1224`** — deprecated `datetime.utcnow()` replaced with `datetime.now(timezone.utc).replace(tzinfo=None)`, deliberately **naive** to match `utcnow()` 's exact semantics. See follow-ups.
- **`AMPLITUDE_SCALE.get()` fallback inconsistency** — called with two different defaults across five sites: `np.sqrt(2)` in the live path (`collector.py:231, :550`) and `1.0` in the reload path (`collector.py:992, :1147, monitor/controller.py:267`). An unrecognised mode reconstructed a loaded trend **1.414x off** relative to the live trend on the same plot. All five now call `util.amplitude_scale()`; the fallback is `'0-P'` because every call site already normalises with `or '0-P'` before the lookup. Unknown modes are logged. A test asserts no `AMPLITUDE_SCALE.get()` survives anywhere in `src/`.
- **`util.py`** — no `__all__`, so `rev80/__init__.py` 's `from rev80.util import *` re-exported `np` and every imported name into the top-level namespace. Added an explicit `__all__`. `data_dir` is listed deliberately: it is imported into `util` from `rev80._paths` and reached as `rev80.data_dir()` by six call sites in `gui.py / headless.py`, so omitting it would have broken them at runtime rather than at import.
- **18 ruff errors** cleared (unused imports/variables, one `E401`). The five unsafe `F841` s were handled by hand: `test_vibechecker.py:186` 's `old_sr` was kept and *asserted on* — `test_load_offline_adjusts_maxfreq` captured the pre-load samplerate but never compared against it, so the “adjusts” behaviour it is named for went unverified. No `noqa` added anywhere.

Reverted

- **`.python-version`** — briefly changed to `3.13` on the premise that the literal `vibecheck` was a stale string. That premise was wrong: `vibecheck` is a real `pyenv-virtualenv` holding every project dependency, and naming a `virtualenv` there is correct `pyenv-virtualenv` usage. The change resolved to a bare interpreter with no packages and produced 15 collection errors. Reverted; the `pyproject.toml` changes from the same commit stand.

refactor/event-pipeline (develop)

Changed

- **`collector.py`** — removed `callbacks` dict entirely; all consumers now read from `frame_cache` via `new_frame_event` rather than receiving samples directly
 - `collect_sample()` rewritten: `new_frame_event.clear() / wait() / frame_cache[-1]` instead of a `_one_shot` closure pinned into `callbacks`
 - `_data_callback()` simplified: appends to cache and sets event; no callback fan-out
- **`collector.py`** — `data_callback` renamed `_data_callback` (internal-only convention)
- **`gui.py`** — internal methods renamed with leading underscore: `display_frame` → `_display_frame`, `poll_new_frames` → `_poll_new_frames`, `create_gui` → `_create_gui`

- **gui.py** — removed vestigial `callbacks['plots']` registrations from `_on_load_file`; file-load display now goes through `new_frame_event` → `_poll_new_frames` exclusively, eliminating a latent DPG thread-safety bug (hardware-thread `_display_frame` call)
- **gui.py** — trend plot (`_update_trend_plot`) now called unconditionally in `_display_frame` so loaded HDF5 trend data is rendered in offline browse mode
- **gui.py** — removed two merge-artifact duplicate method definitions: `poll_new_frames` (stale single-quote copy) and `_on_save_click` (old DPG dialog version)
- **gui.py** — removed duplicate `ACQ_NOTES` widget block in `_create_gui` (caused DPG “alias already exists” crash on `test_gui_build`)

Refactored

- **Tests** — `callbacks` references replaced with `frame_cache` reads across `test_vibechecker.py`, `test_multichannel.py`, `test_picoscope.py`, `test_scope_sensor.py`

hotfix/hpf-integration-fix (2026-08-28)

Fixed

- **collector.py** — `process_sample()` highpass + integration blow-up (field-reported spurious spike at ~1–2 Hz)
 - Root cause 1: the integration transfer function ($(2\pi f)^{-n}$, applied at all three sites — 5-order overalls, PSD, time-domain output) only zeroed the exact DC bin; bin 1 (1x binsize) received the full multiplier applied to residual near-DC energy and dominated the whole spectrum (0.313 in/s at bin1 vs. a real 0.747 in/s tone peak in a captured reference file)
 - A first fix attempt weighted the transfer function by the highpass filter’s frequency response (`scipy.signal.sosfreqz`); this suppressed bin1 by ~50,000x but, being a smooth multiply across many bins rather than an exact single-bin removal, behaved as a wide kernel under circular convolution and badly distorted the time-domain signal at both block edges
 - Fixed instead by hard-zeroing bin1 directly, unconditionally (not gated on `highpass_enabled`) — zeroing an exact FFT bin removes one Fourier basis component losslessly with no boundary sensitivity, the same property that already made the bin0 zero safe
 - Root cause 2 (“edge wobble”, found while investigating #1): the zero-phase highpass (`sosfiltfilt`, introduced in feature/anti-alias) effectively doubles filter order via its forward+backward pass, overshooting the raw signal by 35–45% at both block edges for a low cutoff over a short block (10 Hz over a 1s/4096-sample block); reverted the highpass filter back to causal `sosfilt` (~8–10% startup transient)
 - Verified against real hardware captures (`DEVDATA/hpf-10hz.h5`, 78 Hz / 0.75 in/s-OP test tone): bin1 now exactly 0.0, overall amplitude 0.760 in/s vs. an expected ~0.75, residual edge softness reduced from 350%/246% to ~30–55%
 - **Tests** — 4 new regression tests in `test_sample.py`: bin1 hard-zeroed for integration regardless of `highpass_enabled`, bin1 left untouched for differentiation and passthrough, and a bound on time-domain edge overshoot for the causal highpass
-

feature/anti-alias (2026-08-26)

Added

- **picoscope.py** — mandatory anti-alias oversample/decimate stage in `PicoScopeStream`
 - Field incident: a high-frequency bearing-fault harmonic aliased into the low-frequency band at low apparent power, injecting spurious spectral energy; root cause was driving the ADC directly at the target analysis rate with no anti-alias filtering
 - Always oversamples the ADC (`effective_osr` , up to 4x, capped by `STREAMING_CEILING_HZ` — a ceiling measured on real hardware: continuous `ps4000aRunStreaming / GetStreamingLatestValues` silently drops most samples above ~100–250 kHz depending on channel count, with no error indication), applies a zero-phase anti-alias filter, then decimates back to `config.samplerate` via the new `antialias_decimate()` helper — fully transparent to `DataCollector` and everything above it
 - Electrically verified on a PicoScope 4424A with siggen loopback: a tone above target Nyquist that aliased to a spurious 125 mV peak under naive decimation is suppressed 61.9 dB by the real pipeline; an in-band tone near maxfreq passes through unattenuated
- **picoscope.py** — second, independent streaming-rate watchdog flags (`.degraded`) sustained USB throughput below `_RATE_DEGRADED_THRESHOLD` of the requested raw rate — the same silent data-loss mode uncovered during the anti-alias investigation, invisible to the existing silence watchdog since callbacks keep firing with `status='OKAY'` ; deliberately never triggers `_try_recover()` (a USB/bus bandwidth ceiling, not a device hang)
- **sample.py** — `VibeSample / ChannelResult` gain a `degraded: bool` field
- **tests/test_antialias.py** — regression tests for `antialias_decimate()` : aliasing tone suppressed >20x vs. naive decimation, factor=1 no-op, legitimate low-frequency tone survives intact, independent multi-channel handling
- **tests/test_picoscope.py** — `TestRateDegradationWatchdog` : monkeypatched-clock coverage of `_check_rate_degradation()` (healthy/degraded/recovery), confirms `_try_recover()` is never called for this condition

Changed

- **sample.py / util.py** — `samplerate` now derives from `nextpow2(2.56 * maxfreq)` instead of 2x, guaranteeing $\geq 28\%$ Nyquist margin for the anti-alias filter's transition band (matching the ratio commercial FFT vibration analyzers use); `MAXFREQ_PRESETS` drops the 100k/250k/500k Hz entries — hardware measurement showed these already exceed this PicoScope's continuous-streaming ceiling, silently corrupting captured data
- **collector.py** — `receive_data()` threads the `degraded` flag from each incoming frame onto every channel's `VibeSample / ChannelResult` and persists it into saved `.h5` files alongside `overflow` ; new `DataCollector.stream_degraded` property mirrors `is_streaming`
- **collector.py** — `process_sample()` 's lowpass Butterworth block removed (anti-aliasing is now mandatory upstream in `PicoScopeStream`); highpass filter switched from causal `sosfilt` to zero-phase `sosfiltfilt` (reverted in hotfix/hpf-integration-fix, below)
- **gui.py** — Lowpass checkbox/field removed from the acquisition dialog; status line shows the auto-derived AA cutoff instead of “LP ...” text; a warning line appears when the stream is degraded

Removed

- User-facing `lowpass_enabled / lowpass_fc` fields — they ran after the ADC had already sampled and so could never actually prevent aliasing; anti-aliasing is now mandatory and handled at capture time

rebrand-to-rev80 (2026-08-19 – 2026-08-28)

From this point forward the product is named **Rev80** (formerly vibechecker). Earlier sections below retain the historical “vibechecker” name as an accurate record of the codebase at the time.

Changed

- Rebrand vibechecker → Rev80 across the codebase, packaging, and docs
 - Package moved from `./vibechecker/` to `./src/rev80/` (src-layout; history preserved via `git mv`); all imports updated from `vibechecker` → `rev80`
 - `pyproject.toml`: package name `rev80`, entry points `rev80 / rev80-headless`, `where=["src"]`, `pythonpath=["src"]`
 - User data dir moved to `~/Documents/Rev80/`; config dir moved to `~/.config/rev80/`
 - GUI title/label, CLI `prog` name, installer (`installer/rev80.iss`), build spec (`rev80.spec`), and icons (`assets/icons/rev80.ico / .svg`) renamed to match; README/PROGRESS docs updated
 - `_paths.py` fix: `resource_path()` 's base needed one more `.parent` after the src-layout move added a directory level (it was resolving into `src/` instead of the project root); `logger.py` now resolves `logging.yaml` directly relative to its own file instead of through `resource_path()`
- `build/collect_pico_dlls.py` / `build.sh` — search a bundled `vendor/` folder for PicoSDK DLLs first, removing the requirement to install PicoSDK system-wide before building
- Repo root reorganized: loose docs (`CHANGELOG.md`, `PROGRESS.md`, `VibeGui Project.md`) moved into `doc/`; dev scripts (`build.sh`, `render_progress.sh` → `render_md.sh`, `runtimes.ipynb`) moved into `scripts/`; `rev80.spec` moved into `build/` alongside `collect_pico_dlls.py`, with `ROOT` updated to resolve from `SPECPATH` 's parent so spec-relative paths still point at the repo root; stale `requirements.txt` and `picosdk-install.md` removed
- `.gitignore` — corrected the PyInstaller build-output path exclusion left stale by the repo reorg
- Project management docs revisited — outstanding requirements and tech debt items reviewed and updated

hotfix/welch_leakage (2026-08-18)

Fixed

- `collector.py` — Welch window functions (Hann, Blackman-Harris, etc.) introduce spectral leakage that inflates the reported overall vibration level by a window-dependent factor (Hann: $\sqrt{3/2}$); overall amplitude is now derived from the RMS of an exact-inverse time-domain reconstruction instead of summing spectral peaks
 - The 5-order ($-2 \dots +2$) mV RMS overalls are now computed via `irfft` of the integration-scaled `rfft` — reusing the same plain, unwindowed `rfft` already computed once for the time-domain output step, so the round-trip is lossless — as `sqrt(mean(time_ord**2))` instead of `sqrt(sum(psd_ord**2))`
 - The target-unit overall (step 7) now reuses the cached 5-order mV RMS column instead of re-deriving it from the spectrum
- `collector.py` — `process_sample()` now guards against the resolved integration order falling outside `[-2, 2]`; previously an out-of-range order silently indexed the wrong overalls column, producing false scaling — it now logs an error and returns `None` for the frame

feature/monitor_mode (2026-05-28 – 2026-06-30)

Added

- **monitor/ package** — Monitor Mode interval datalogger
 - `gate.py`: `IntervalGate` with snap-to-grid scheduling and burst mode
 - `session.py`: `MonitorSession` frozen dataclass (later gains `acq_snapshot / channel_snapshot / sensor_snapshot`, `cooldown_enabled / cooldown_s`)
 - `anomaly.py`: `AnomalyHook` protocol + `NullAnomalyHook` stub
 - `writer.py`: `MonitorWriterThread` (daemon, disk-space guard)
 - `controller.py`: `MonitorController` orchestrating gate/writer/anomaly
 - `gui.py`: Monitor card with config tab and arm/disarm controls (relabeled and reworked repeatedly through the branch — see Changed)
 - 51+ new tests across `test_monitor_gate.py`, `test_monitor_index.py`, `test_monitor_controller.py`
- Session storage iterated through several revisions in-branch:
 - v4: monitor captures write the standard metadata+frames HDF5 layout so `collector.load_data()` can open them directly with no adapter; a `capture_trigger` root attr distinguishes monitor files from manual saves
 - v5 (Phase 2 storage redesign): one `session.h5` per session (`DEVDATA/monitor/{session_id}/`) replaces per-capture files and the SQLite index entirely — `/monitor/{N}/` groups for interval captures, `/burst/{burst_id}/{frame_index}/` groups for burst events, `/burst.attrs['burst_list']` JSON for fast browser rendering without loading frame data
- **Burst capture** — manual (`trigger_burst()`) and anomaly-triggered bursts with a pre-trigger ring buffer
 - Session browser rewritten as a two-tab modal (Monitor / Burst views); `resize_frame_cache()` now expands the cache before load so all captures in a session are browsable, not just the most recent `cache_frames`
 - A run of alignment fixes during the branch: the trigger frame was double-counted in the pre-trigger snapshot; `_burst_all_results` indices were misaligned against pre-trigger frames; burst `rel_time` rebased to the trigger frame ($t=0$) instead of session-relative time; frame cache resized before loading burst frames (previously silently evicted pre-trigger frames at the default 32-frame cache); pre-trigger overalls pre-computed at capture time (`_compute_pretrigger_overalls()`) rather than reprocessed at load; disk-usage estimate corrected to include the pre-buffer, not just burst duration; trend unit conversion restored when browsing a loaded session (sensor wiring was missing on the session-load path, so channels fell back to raw mV)
- **Anomaly detection hooks** (`monitor/anomaly.py`)
 - `RmsThresholdHook` — EWMA self-calibrating per-channel baseline, triggers when `|current - baseline| / baseline` exceeds a threshold for N consecutive frames; tracks streak onset time so `trigger_time / trigger_rel_time` reflect anomaly onset rather than the confirmation frame
 - `SpectralThresholdHook` — initially a stored-baseline bin-by-bin dB comparison with an optional fmin/fmax band; later rewritten to an EWMA per-bin baseline with a percentage threshold and peak-frequency reporting in the trigger reason
 - `CompositeAnomalyHook` — tries each hook in order, returns the first event; propagates `reset_baseline()` to all children
 - `FixedThresholdHook` — independent upper/lower level triggers with unit conversion via `UNIT_TO_SI`
 - `ewma_alpha_from_time(tau, dt)` helper lets EWMA settings be specified as a time constant τ (seconds) instead of the opaque `alpha` value; GUI shows a live computed- α label

- Post-burst cooldown gating replaces arm/disarm entirely: the anomaly hook is supplied once at `start()` and active for the whole session; a cooldown deadline set after any burst fires suppresses further triggers while interval captures continue normally
- GUI: Anomaly Detection section in the Monitor config tab (hook-type combo, RMS/Spectral/Fixed-level/cooldown settings groups, tooltips throughout); “Reset Baseline” replaces the old Arm/Disarm button
- 15+ new tests (`test_monitor_anomaly.py`) covering warmup gating, consecutive-N triggers, baseline set/reset, composite fallthrough, streak tracking, cooldown
- **Headless CLI** (`headless.py`) — interval datalogger without the GUI
 - `python -m vibechecker.headless / vibechecker-headless` console script: discovers a PicoScope (or `--device sim`), loads saved device config, starts the stream, runs `MonitorController`, prints periodic status, and shuts down cleanly on SIGINT/SIGTERM with a session summary
 - `--headless` flag routes `__main__.py` into headless mode; shares the collector/monitor/sample/picoscope pipeline unchanged with the GUI path
 - `--from-file` loads an `.h5` file or monitor session directory on startup (both GUI and CLI); `--[no-]autodetect` controls device discovery, defaulting off when `--from-file` is given
 - `--init-config` seeds `~/.config/vibechecker/` with `acquisition.yaml` and `devices/picoscope-defaults.yaml`
 - Monitor/anomaly settings configured once in the GUI persist to device config and are read back by headless as defaults, overridable by CLI args
 - `dearpygui` made an optional dependency (`pip install -e .` for headless-only installs; `pip install -e '[gui]'` for the GUI)
- **drivers/install-picoscope4000a-driver.sh** — Linux PicoScope driver install script; registers `/opt/picoscope/lib` with `ldconfig` after install
- **gui.py** — Frame info card in the right panel: timestamp, block size, and sample rate always shown; burst-browse-only fields (frame time relative to trigger, burst ID, trigger type/timestamp, max overall per channel); session ID when browsing a session or burst
- **gui.py** — CommitMono Nerd Font icons throughout the left panel (card headers, action buttons, browse-nav arrows); font auto-downloaded on first build and bundled into the frozen app
- **gui.py** — global keyboard shortcuts: Ctrl+A autoscale, Ctrl+K start/stop, Ctrl+S save, Ctrl+O load, Ctrl+Q quit, ←/→ frame browse (live only)
- **gui.py** — configurable frame cache depth (`AcquisitionSettings.cache_frames`, default 32) with derived recording-window and memory-usage display

Changed

- **config.py** — layout split into `acquisition.yaml` (maxfreq/binsize/monitor/anomaly, per instance), `devices/picoscope-<model>-<SN>.yaml` (channels + siggen, per device), and `devices/picoscope-defaults.yaml` (new-device template); `device_config_path()` now takes (`model_name`, `serial_number`) and produces human-readable filenames; no migration path — delete `~/.config/vibechecker` to reseed
- **collector.py** — overalls now always computed over the full FFT spectrum; the `trend_fmin / trend_fmax` “Trend Frequency Window” band-limiting knob removed entirely (dataclass field, config default, UI tags, dialog widgets, tests)
- **collector.py** — `nperseg` clamped to signal length in the Welch PSD call to avoid spurious warnings on short blocks
- **collector.py** — `_load_v3 / _load_v4` consolidated into a single `load_data()` method, fixing a v4 load crash where the old `_load_v4` delegated to `_load_v3`, which read a `data` key that doesn't exist on v4 trend groups

- Version tooling: git-describe version written by a `post-commit` hook, later moved to `pre-commit` so `_version.py` is included in the commit that changes it; installer version now derived from `_version.py` at build time instead of hardcoded in the `.iss` file
- Build fixes accumulated through the branch: UPX disabled (was corrupting the frozen `python3XX.dll`'s PE import table); `pandas` removed (pulled in a `pytz` version-detection failure in frozen builds; the peaks table now uses `list[tuple]`); `plyer`'s Windows filechooser dependency (`win32com / pywintypes`) added to hidden imports; `pyyaml` hidden-import name corrected (`yaml`, not `pyyaml`); `dist/` cleaned and any running instance killed before rebuilding; `assets/` bundled into the frozen app (font was missing, causing a startup crash); PyInstaller cache wipe no longer runs by default (`./build.sh all clean` to force it); Inno Setup arch identifier updated to `x64compatible`

Removed

- `monitor/index.py` (SQLite session index) — superseded by the single- `session.h5` v5 layout
- Arm/disarm as a user-facing concept — anomaly detection now runs for the whole monitored session, gated only by post-burst cooldown

refactor/mv-domain-trend (2026-05-05)

Changed

- **`sample.py` / `collector.py`** — `VibeSample` now stores raw mV throughout; all sensitivity conversion, Butterworth filtering, and Welch PSD computation moved into `DataCollector.process_sample()`
 - `VibeSample: process(), _convert_time_domain(), push_sample(), save(), load()` removed; adds `overflow: bool` and cached `psd_mv / freq_hz / _psd_config_key / overall_ampl_by_integration_order (5,)` mV RMS fields
 - `ChannelResult` gains an `overflow` field
 - `receive_data()` becomes a pure mV pass-through (no sensitivity or filtering applied)
 - `process_sample(ch, sample): filter → Welch PSD (cached) → 5-order overalls (cached) → sensitivity + SI + integration → ChannelResult`
 - `process_samples()` becomes the collector-owned frame dispatcher; appends to trend only while streaming, uses a `-1` cursor index in browse mode
 - Trend storage changed from a dict-of-lists to `dict[int, {rel_times: ndarray, orders: ndarray(M,5)}]`; new `get_trend_for_display()` centralizes unit/sensitivity/amplitude-mode conversion so `gui.py` no longer imports `util` for it
 - `get_active_eu()` returns `'mV'` immediately when no `ScopeSensor` is assigned, avoiding nonsensical unit conversion before sensitivity is known
 - HDF5 v4: per-channel `(M,5)` orders matrix plus per-channel `rel_times`; v3 files are promoted to v4 trend structure on load (order-0 only)
- **`gui.py`** — display loop simplified around `process_samples() / get_trend_for_display()`; `_compute_channel_result()` removed; overflow now read from `result.overflow` instead of a bitmask
- **`picoscope.py`** — `_setup_siggen()` now also called from `start()`; previously it only fired on the reconnect path in `_try_recover()`, so the signal generator never started on the initial `stream.start()` call
- **`collector.py`** — overalls now computed over the full FFT spectrum unconditionally (groundwork later formalized by removing `trend_fmin / trend_fmax` in `feature/monitor_mode`); HDF5 load paths consolidated

Added

- **gui.py** — global keyboard shortcuts (Ctrl+A/K/S/O/Q, arrow-key frame browse) — later extended in `feature/monitor_mode`
- **gui.py** — configurable frame cache depth with recording-window/memory display, `DEFAULT_CACHE_FRAMES` sourced from `config._BUILTIN_DEFAULTS` — later extended in `feature/monitor_mode`
- Build tooling: git-describe version written by a post-commit hook and embedded into the installer via `_version.py`; PyInstaller/Inno Setup fixes for UPX corruption, a `vibechecker.spec` merge conflict, `pandas / plyer` bundling, and `pyyaml` hidden-import naming

feature/windows-build (2026-03-29 – 2026-05-27)**Added**

- **_paths.py** — cross-platform path sanitization (Phase 1): `sys._MEIPASS`-aware `data_dir()`, `log_dir()`, `resource_path()`; replaces the third-party `path` library with stdlib `pathlib` across all modules; `SAVEDIR / DataCollector.datadir` route to `~/Documents/vibechecker/data/` in frozen builds
- **_pico_loader.py** / `build/collect_pico_dlls.py` — PicoScope driver bundling (Phase 2): a Windows script locates `ps4000a.dll / picoipp.dll` from the PicoSDK install (registry + default paths), validates the 64-bit PE header, and copies them to `drivers/`; `_pico_loader.py` registers that directory via `os.add_dll_directory()` before `picosdk` import, handling both frozen (`sys._MEIPASS/drivers/`) and dev layouts
- **vibechecker.spec** / `installer/vibechecker.iss` / `build.bat` / `build.sh` — PyInstaller + Inno Setup build pipeline (Phase 3): one-dir PyInstaller build bundling `logging.yaml`, driver DLLs, and dearpygui data; Inno Setup 6 script for a 64-bit, non-admin install to `%LOCALAPPDATA%` with a soft PicoSDK-present check and Start Menu/desktop shortcuts
- App icon: placeholder icon wired into both the PyInstaller spec and Inno Setup script (real artwork deferred)
- **README.md** — restructured for users, contributors, and Windows builders: Windows install section (PicoSDK + installer), cross-platform source install section, Contributing section (dev env, project layout, icon swap guide), full prerequisites table (Python, Git for Windows, PicoSDK, Inno Setup)

Fixed

- PicoSDK 11.x detection by DLL path when the registry key is absent
- `logging.yaml` moved into `vibechecker/` so `resource_path()` resolves it correctly in both dev and frozen builds
- Removed `scipy` submodule excludes from the PyInstaller spec — `scipy.signal` depends on `scipy.linalg` internally and the build broke without it
- Debug `print()` statements stripped from `gui.py / __main__.py`; Inno Setup branding (publisher, copyright, license) updated
- Inno Setup Compiler (ISCC) lookup corrected to the proper install `APPDATA` directory
- Icon assets moved to `assets/icons/`

Changed

- **picoscope.py** — `FindPicoScope` replaced with multi-device enumeration via `ps4000aEnumerateUnits`, opening each device by serial number; `PicoScopeStream` gains `_open_unit_by_serial` so it targets the correct device when multiple scopes are connected

- **picoscope.py** — hand-maintained `_channels_for_model` lookup table replaced by `_probe_channel_count`, which calls `ps4000aSetChannel` for channels A–H and counts successes, so any future hardware variant self-reports its channel count without a code change
- Ruff lint fixes across `picoscope.py`, `sample.py`, `util.py`

feature/channel-naming (2026-04-03 – 2026-04-13)

Added

- **sample.py** — `AcquisitionSettings` gains `channel_names` and `channel_target_units` dicts with `name_for(ch)` / `target_unit_for(ch)` helpers, persisted in `to_dict` / `from_dict`; later extended with `channel_amplitude_modes`, `channel_couplings`, `channel_voltage_ranges` dicts and matching typed accessors (`amplitude_mode_for`, `coupling_for`, `voltage_range_for`)
- **gui.py** — offline post-analysis mode: `load_data` auto-configures `enabled_channels` and `maxfreq` from file contents; plot series, axes, and results panel sync after load; connection summary shows “File Loaded” (yellow) with frame/channel count; trend plot gets a vertical cursor line for the browsed frame position; acquisition buttons disabled when no device is connected
- **gui.py** — native file dialogs via `plyer.filechooser` (kdiallog on KDE, native Win32) replace the DPG file dialog; `SAVEDIR` resolved to an absolute path so the dialog opens in the right place; browse-waveform controls consolidated into a single `_on_browse` dispatcher
- **gui.py** — Channels tab redesign: per-channel collapsing header row (color swatch + name text, then coupling/range/sensor on a second line), themed colored-when-enabled / grey-when-disabled; enabled state moved outside the collapsing header with a target-unit/amplitude indicator on the header itself; amplitude-mode combo added per channel; color indicators switched from a `■` glyph (didn’t render in the app font) to `drawlist` / `draw_rectangle`
- **util.py** — expanded `UI_Elements` tag registry: named constants replace previously hardcoded `DEVSETUP_*` / `SREG_FIELD_*` strings; new tags for the redesigned channel rows (`scope_ch_name_text`, `scope_ch_hdr_theme`, `scope_ch_amplitude_mode`)
- File Handling card gains a multiline Measurement Notes widget, read on save and populated on load
- Tooling: `ruff` added to dev dependencies with a `.github/hooks/pre-commit` hook (`git config core.hooksPath .github/hooks` to activate); `pyproject.toml` sets line-length 120, ignores E402/E701

Changed

- **collector.py** — HDF5 format migrated v1 → v2 → v3 across the branch
 - v2: metadata moved into `.attrs` (`/acquisition`, per-frame/per-channel groups) instead of child datasets; `_load_v1` retained for back-compat
 - v3: structured `/metadata/` group — `/metadata/acquisition` (scalars only), `/metadata/scope_sensors/{id}` (sensor library, each unique sensor stored once), `/metadata/channels/{ch}` (name/unit/coupling/voltage_range/sensor_id/target_unit/amplitude_mode); `/frames/{i}/{ch}/data` stores raw samples only; `/trend/re_l_times` becomes a shared axis with `/trend/{ch}/data` per channel; `load_data` dispatches to `_load_v3` only, `_load_v1` / `_load_v2` removed; `save_data` overwrites existing files instead of early-returning
 - Fixed a file-mode bug where the channel list in the config dialog was derived from `enabled_channels` rather than the frame cache, so disabling a channel made it disappear from the dialog entirely
- **scope_sensor.py** — `amplitude_mode` field removed; it’s a per-channel acquisition setting, not a sensor property, and moves to `AcquisitionSettings.channel_amplitude_modes` (old files with the key are silently ignored on load)

- **gui.py** — config dialog scroll fix: outer window gets `no_scrollbar / no_scroll_with_mouse`, the tab bar and each tab's content are wrapped in their own `child_window` so scrolling stays contained per-tab, and the Close button is pinned outside the tab area
- **sample.py** — spectral peak detection: display count default raised (3 → 6) and relabeled "Peak Display"; minimum peak distance changed from a spectrum-length-relative value to a fixed 5 bins

refactor/queue-handoff (2026-04-08)

Changed

- **collector.py / gui.py** — decouple collector → GUI with `threading.Event`, precursor to the fuller refactor/event-pipeline cleanup above
 - `DataCollector.new_frame_event`: set by `data_callback` and `reprocess_last_block`
 - `GUI.poll_new_frames`: polls the event each render tick and grabs `frame_cache[-1]`
 - Switched from `dpg.start_dearpygui()` to a manual render loop
 - Removed the `callbacks['plots']` registration — the GUI no longer hooks directly into the collector for display; `collect_sample`'s one-shot capture still used the `callbacks` dict at this point (removed entirely later, in refactor/event-pipeline)
 - Naturally handles GUI lag: rendering always shows the latest frame and skips intermediates, while all frames remain in `frame_cache` for browsing regardless of render rate
- **gui.py** — guarded `poll_new_frames` against a rare `IndexError` race if the hardware thread shifts the deque between `len()` and indexing; silently skips the frame since fresh data arrives next tick
- **README.md / CLAUDE.md** — architecture docs updated for the event-based collector↔GUI decoupling: callback diagrams replaced with the event-signal + poll-loop pattern; stale `sounddevice` references removed

feature/picoscope

Added

- **vibechecker/picoscope.py** — PicoScope 4000A acquisition backend (Phase 1)
 - `FindPicoScope()` — enumerates connected PS4000A units; returns `VibeSensor`-compatible dicts with `unit=['mV']`, mirroring the `FindDigiducer` interface
 - `PicoScopeStream` — background polling thread wrapping `ps4000aRunStreaming`
 - Converts ADC counts → mV via `adc2mV` on every driver callback
 - Accumulates variable-sized chunks into exact `blocksize` blocks before firing `DataCollector.receive_data`
 - Implements `.active / .start() / .stop() / .close()` interface (compatible with `sounddevice.InputStream` and `SimulatedSensor`)
 - Handles USB-only / non-USB3 power states (status codes 282 / 286)
 - Reads back actual achieved sample rate after `ps4000aRunStreaming` and updates `AcquisitionSettings.samplerate`
- **AcquisitionSettings** — two new PicoScope-specific fields (`sample.py`):
 - `voltage_range: int = 8` — PS4000A range index (8 = PS4000A_5V)
 - `coupling: str = 'AC'` — channel A input coupling ('AC' or 'DC')
- **util.py** — extended `SAMPLERATES` list to include PicoScope-relevant rates: 100 kHz, 200 kHz, 500 kHz, 1 MHz
- **util.py** — added `'mV'` to `SUPPORTED_UNITS / UNITS` dict for raw voltage passthrough

- `examples/ps4000a_triangle_stream_plot.py` — standalone script: generates a 500 Hz triangle wave (0.5 V amplitude, +1.4 V DC offset) via the PS4000A signal generator, streams Channel A at 50 kHz for 100 ms, then renders a Plotly HTML report with Welch PSD (10 windows, 50 % overlap) and top-5 peak detection

Changed

- `sensor.py` — `VibeSensor.find()` now calls `FindPicoScope()` instead of `FindDigiducer()`; `VibeSensor.connect()` returns a `PicoScopeStream` for hardware sensors and a `SimulatedSensor` for the simulation path
- `sensor.py` — removed `sounddevice` import and the `sounddevice` reset workaround; renamed internal `_callback` → `_sd_callback` (simulation path only)
- `collector.py` — removed `sounddevice` import; broadened `PortAudioError` catch in `start_stream()` to `Exception`; rewrote `recieve_data()` channel extraction to handle both `(N, channels)` 2-D arrays and 1-D arrays, with channel index clamping
- `sample.py` — `VibeSample.get_accel()` wraps `convert_units` in a try/except so unsupported conversions (e.g. 'mV' → 'g' before sensitivity is applied) pass through raw data instead of raising
- `README.md` — added PicoScope Integration section: architecture change, new `AcquisitionSettings` fields, Phase 1 data flow diagram, Phase 2 roadmap

Removed

- `digiducer.py` / `sounddevice` no longer used in the main acquisition path (file retained for reference; `FindDigiducer` still exported from `__init__.py`)

[0.0.1] — early prototype (2025, sounddevice/Digiducer path)

Initial public snapshot of the **sounddevice / Digiducer** acquisition path with:

- `VibeSensor` / `SimulatedSensor` / `DataCollector` pipeline
- `VibeSample` with Welch FFT, velocity spectrum, HDF5 save/load
- `AcquisitionSettings` with enforced interdependencies
- `dearpygui` GUI with real-time time-domain and frequency-domain plots
- Butterworth highpass filter (4th-order SOS, default 10 Hz cutoff)
- Simulated bearing-defect signals (`GenerateBearingVibration_SpectralMethod` , `GenerateBearingVibration_TemporalMethod`)
- Comprehensive README and pytest suite